Report on

SMARTMAIL ASSISTANT

Submitted to:

University Institute of Computing

Submitted by:

Rishi Soni

Chandigarh University, Mohali BONAFIDE CERTIFICATE

Certified that this project report on **SMARTMAIL ASSISTANT** is the bonafide work of **Rishi Soni**, carried out as a project under the supervision of the undersigned.

SIGNATURE	SIGNATURE
SUPERVISOR	HEAD OF THE DEPARTMENT
Name: _____	University Institute of Computing
Designation: _____	Chandigarh University

Submitted for the project viva-voce examination held on: _____

INTERNAL EXAMINER	EXTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION **1-2**

1.1 Need Identification 1

1.2 Identification of Problem 1

1.3 Identification of Tasks 1-2

1.3.1 Functional tasks 1

1.3.2 Engineering and documentation tasks 2

1.4 Timeline 2

1.5 Organization of the Report 2

1.6 Scope boundary and operating assumptions 2

CHAPTER 2: BACKGROUND STUDY **3-5**

2.1 Timeline of the Reported Problem 3

2.2 Proposed Solution 3-4

2.3 Bibliometric Analysis 4

2.4 Review Summary 4

2.5 Problem Definition 5

2.6 Goals / Objectives 5

2.7 Why Use These Technologies Over Other Options 5

CHAPTER 3: DESIGN AND PROCESS **6-9**

3.1 Evaluation and Selection of Specifications / Features 6

3.2 Design Constraints 6

3.3 Design Flow 7

3.4 Creation of SmartMail Assistant 7-8

3.4.1 React and Material UI client 7

3.4.2 Spring Boot and Gemini service 8

3.4.3 Gmail extension 8

3.5 Logo Designing and Naming 9

3.6 Implementation Plan / Methodology 9

3.7 Design decisions and implementation boundary 9

TABLE OF CONTENTS (CONTINUED)

CHAPTER 4: RESULT ANALYSIS AND VALIDATION **10-11**

4.1 Implementation of Solution 10

4.1.1 Report Preparation 10

4.1.2 Project Management and Communication 10

4.1.3 Testing / Characterization / Interpretation / Data Validation 11

4.1.4 Acceptance criteria and evidence handling 11

CHAPTER 5: CONCLUSION AND FUTURE WORK **12-13**

5.1 Conclusion 12

5.2 Future Work 12-13

5.2.1 Improve prompt and response handling 12

5.2.2 Strengthen privacy and security 12

5.2.3 Make the Gmail integration more reliable 12

5.2.4 Expand validation and usability 13

5.2.5 Complete deployment and accessibility work 13

5.3 Prioritized implementation roadmap 13

REFERENCES **14**

APPENDICES **15-70**

A. Source code and system walkthrough 15-19

B. Operations and user guide 20-23

C. Application screenshots 24-25

D. API, data handling, and security 26-30

E. Extended validation plan 31-36

F. Maintenance and handover 37-39

G. Project requirements and design record 40-45

H. Interaction and component design 46-51

I. Detailed validation and evidence worksheets 52-57

J. Deployment, quality, and handover reference 58-61

K. API implementation reference 62

L. Implementation and release analysis 63-70

ABSTRACT

SmartMail Assistant is an AI-assisted application for preparing draft replies to existing email messages. The project addresses the routine effort involved in writing acknowledgements, confirmations, clarifications, and short updates. It accepts the text of an incoming message and returns candidate wording that the user can review, revise, and copy or place into a draft. The intended outcome is reduced repetitive drafting effort while keeping the sender responsible for accuracy, tone, and the final decision to communicate.

The prototype combines a React and Material UI web interface, a Spring Boot REST service, Google's Gemini generative AI API, and a Chrome Manifest V3 content script intended for Gmail. In the web path, a user enters email content, may select Professional, Casual, or Friendly tone, and activates Generate Reply. The interface checks for blank input, shows a loading state while waiting, displays returned text, and provides a copy action. In the Gmail path, the content script observes page changes, attempts to add an AI Reply control, reads visible message text, requests a reply from the same backend route, and attempts to insert returned text into the compose editor.

The backend provides a common model-access boundary for both clients. A request containing emailContent and tone is bound by the REST controller and passed to a service. The service constructs a prompt, sends a contents-and-parts request to the configured Gemini endpoint, parses the first candidate text, and returns a string to the client. The system therefore demonstrates an end-to-end architecture with separate presentation, API, model-integration, and browser-integration responsibilities. Its current request and response contract remains compact and needs further definition for production use.

The product is designed as a drafting assistant rather than an automatic email sender. Generated text can be fluent yet incomplete or inaccurate, so the user must verify names, dates, amounts, factual

statements, and commitments before using it. The project source does not establish account management, persistent reply history, analytics, attachment processing, or automatic sending. Email content is transmitted through the backend to an external model provider, making data handling and user notice part of the project context.

ABSTRACT (CONTINUED)

Source review indicates that the main web workflow, REST route, Gemini request, and Gmail action are represented in code. The implementation is nevertheless a prototype with integration and operational limits. The React page and Gmail script are separate clients; the extension uses a fixed tone and local endpoint; Gmail interaction depends on page selectors; server-side validation is limited; and the provider-response parser assumes a first candidate and first text part. The controller returns raw text and does not show a complete typed error contract for invalid input, provider failure, or malformed output. The configured local HTTP endpoint is intended for development rather than public deployment.

This report distinguishes observed source behavior from recommendations and planned verification. No executed end-to-end test log, live Gemini output, benchmark, user study, or production deployment record was supplied as project evidence. The included Work 12 screenshot shows the web interface before input, with the generate action disabled. Work 13 shows a Gmail compose view with the AI Reply control. These images demonstrate visible interface states only; they do not prove that a provider response was generated or inserted into a draft.

A complete evaluation should use synthetic or consented email examples and record the source revision, runtime versions, browser, configuration category, observed result, and evidence reference. Deterministic tests can mock the provider to check prompt construction and parsing. Separate integration checks can cover network configuration, timeout behavior, and provider errors. Browser checks should cover loading and recovery states, keyboard access, copy behavior, multiple compose windows, message extraction, draft preservation, and protection against unintended sending. Security review should examine secret handling, request-size limits, CORS, HTTPS, diagnostic logs, and the privacy notice against the actual deployment.

The project contributes a working architectural starting point for integrating generative text assistance into email drafting through both a web page and a Gmail-oriented browser action. The next priorities are to validate fields and tone values on the server, align the base prompt with selected tones, handle empty and blocked model responses, configure endpoints and credentials securely, and make Gmail insertion resilient and accessible. The user should always retain control of the final message. The source supports the feasibility of the proposed workflow, while measured claims about reliability, productivity, and privacy require further implementation and evidence.

LIST OF FIGURES

Fig. 1. Graphical abstract of SmartMail Assistant

Fig. 2. High-level system architecture

Fig. 3. Request and response sequence

Fig. 4. User workflow for generating a reply

Fig. 5. Software component view

Fig. C.1. SmartMail Assistant web interface in its initial state

Fig. C.2. Gmail compose view with the AI Reply extension action

GRAPHICAL ABSTRACT

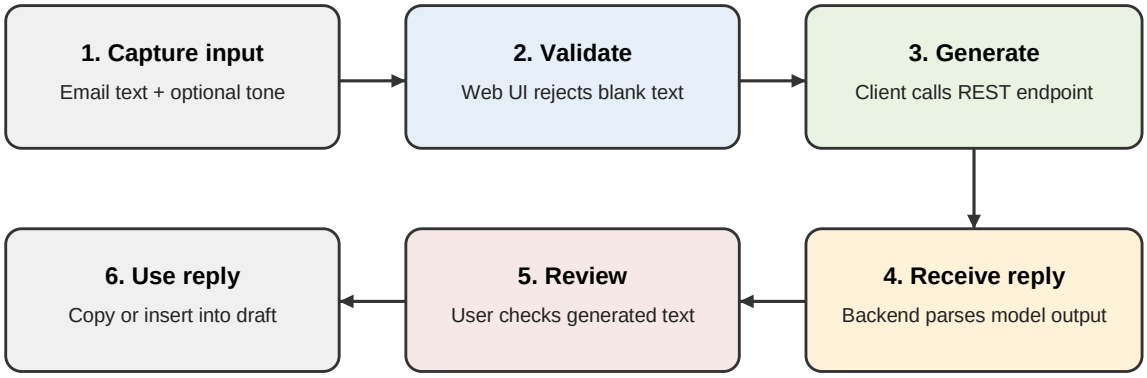


Fig. 1. Graphical abstract of the SmartMail Assistant workflow

The user supplies the original message, chooses or accepts a tone, reviews the generated draft, and decides whether to copy or insert it. The application does not send email automatically.

CHAPTER 1

INTRODUCTION

1.1 Need Identification

Email remains a routine channel for academic, business, and personal coordination. Many messages call for predictable responses such as acknowledging a request, confirming a meeting, asking for clarification, or providing a short update. Repeatedly composing these responses consumes time and can make tone and clarity inconsistent. A drafting assistant can reduce this repetitive effort by proposing text from the message context, while leaving the sender responsible for checking accuracy and deciding whether to send it.

SmartMail Assistant addresses this narrow productivity need. It is designed to work through a small web interface or a Gmail content script. The user provides the message content; a backend prepares a prompt for a generative model; and the resulting text is returned as a draft. A tone choice can guide the wording. The intended benefit is faster first-draft creation, not autonomous correspondence.

Generative output can be fluent while still being inaccurate, incomplete, or inappropriate. For that reason, the interface should present a draft for review rather than sending it. Privacy and security also matter because the original email body is transmitted to an external model service. The user must understand this data path and avoid submitting sensitive information unless the deployment and service terms permit it.

1.2 Identification of Problem

The project tackles the friction between receiving an email and composing a suitable reply. Existing mail clients provide composition tools but do not necessarily create a context-aware reply in the recipient's chosen tone. Copying message text into a separate AI tool introduces context switching and manual transfer. A browser extension can reduce those steps, while a simple web interface can support stand-alone use and demonstrate the backend API.

The implementation also exposes engineering problems that must be addressed before reliable use. The supplied extension reads Gmail through page selectors that may change. It calls a local HTTP endpoint, so it is not ready for remote deployment. The backend accepts the request and calls Gemini, but has no explicit validation layer, rate limiting, authentication, or structured error response. The reply parser assumes a particular response shape. These gaps affect reliability, safe handling of user data, and deployment readiness.

1.3 Identification of Tasks

1.3.1 Functional tasks

- Accept the source email text from the React client or Gmail page.
- Allow the web user to select Professional, Casual, Friendly, or no explicit tone.
- Reject empty web-interface input and communicate loading or failure state.
- Send an HTTP POST request to the Spring Boot API with emailContent and tone fields.

- Construct a model prompt and request a generated reply from Gemini.
- Return the reply text for display, copying, or insertion into the Gmail compose editor.

1.3.2 Engineering and documentation tasks

The engineering work includes identifying the boundaries between clients, API controller, service, and external model; documenting the request contract; reviewing project configuration; identifying operational and privacy constraints; and preparing a validation matrix for future execution. The report also distinguishes implemented behavior from recommended work so that proposed functions are not confused with verified capabilities.

1.4 Timeline

The repository does not contain dated milestone records. The following phase plan is therefore a recommended project sequence rather than a claimed historical schedule.

Phase	Work	Completion evidence
1. Requirements	Define inputs, outputs, tones, and review responsibility	Approved requirement list
2. API foundation	Create request model, controller, Gemini service, and configuration	Local API responds to a valid request
3. Client interfaces	Build React form and Gmail content-script action	Reply displayed or inserted in a test mailbox
4. Validation	Test normal, invalid, error, and edge cases	Recorded test results and resolved defects
5. Hardening	Protect secrets, add HTTPS, controls, and operational monitoring	Deployment checklist and security review

1.5 Organization of the Report

Chapter 1 introduces the need, problem, tasks, and planned sequence. Chapter 2 reviews the project context, proposed solution, technology landscape, and objectives. Chapter 3 explains requirements, constraints, architecture, interface behavior, and implementation method. Chapter 4 records the implementation visible in the supplied source and defines validation cases without claiming they have been executed. Chapter 5 summarizes the work and sets out future improvements.

1.6 Scope boundary and operating assumptions

The prototype accepts text supplied by a user and returns candidate wording for review. The documented scope includes a React web client, a Spring Boot service, a Gemini provider call, and a Gmail-oriented content script. It does not include automatic sending, account management, saved reply history, analytics, attachment processing, or a verified production deployment. Local operation assumes the required runtimes are installed, provider configuration is available to the backend, and both clients can reach the same API. These assumptions should be checked during setup and recorded with any demonstration. Real email content should not be used until the deployment's provider processing, transport, logging, and notice have been reviewed.

CHAPTER 2

BACKGROUND STUDY

2.1 Timeline of the Reported Problem

Email has developed from a store-and-forward messaging protocol into a standard workspace for task requests, approvals, coordination, and customer communication. As the volume and variety of messages increase, users face repeated drafting decisions: what information to include, how formal to sound, and how to keep the response concise. Modern language models make it possible to generate a candidate response from a supplied text prompt, but the result still depends on the quality of the input and requires human review.

The project follows this progression: routine email creates repeated writing work; language generation can propose a draft; an API separates the client from model access; and a browser integration reduces the steps required to use the service. The supplied code implements a basic version of this path with a React client, Spring Boot service, Gemini call, and Gmail content script. It does not include a research dataset or a measured study of time saved.

2.2 Proposed Solution

SmartMail Assistant provides two client paths to a shared generation service. The React application presents a multiline message field, an optional tone selector, a Generate Reply action, a result panel, and a copy control. The Chrome content script adds an AI Reply button to a Gmail compose toolbar, extracts visible message text, calls the same API route, and inserts returned text into the compose editor. Both clients depend on the Spring Boot service, which builds the prompt and forwards it to the Gemini API.

The user remains in control. A generated reply is displayed or inserted as editable draft text; the project source does not send the email. This workflow is summarized in Figure 2 and the interaction sequence in Figure 3.

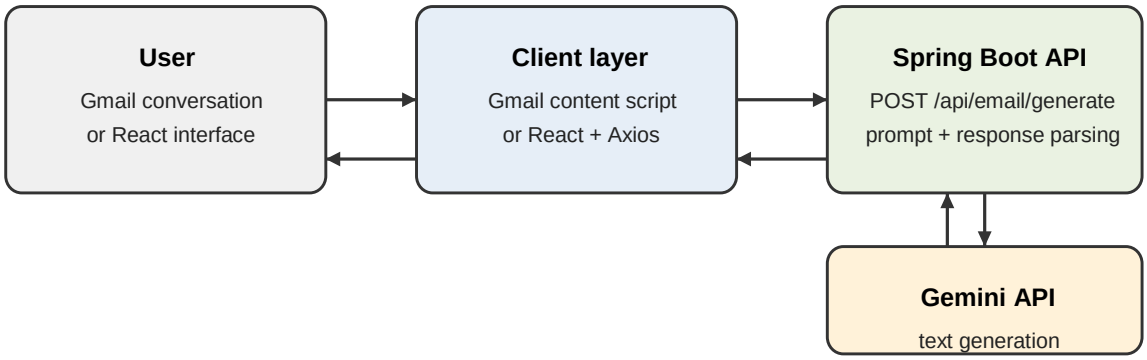


Fig. 2. High-level system architecture

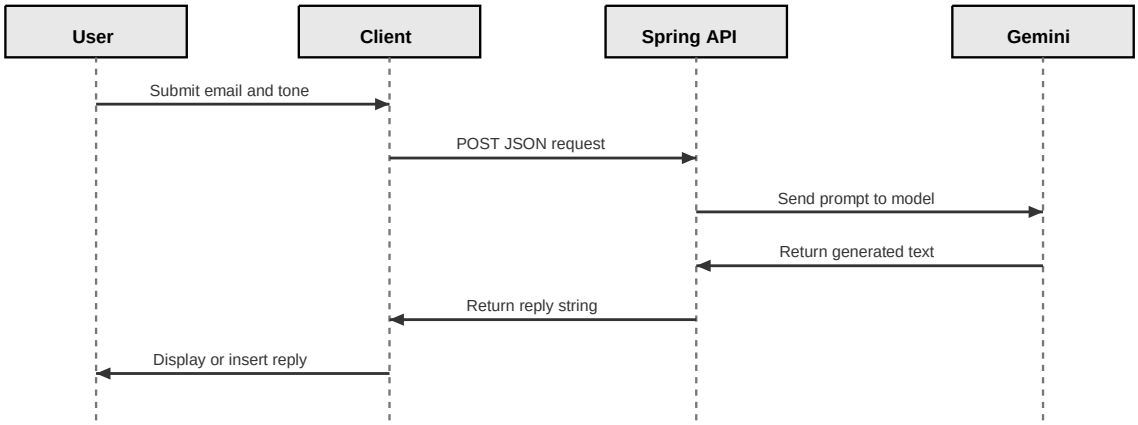


Fig. 3. Request and response sequence

2.3 Bibliometric Analysis

A quantitative bibliometric study requires a defined publication database, search query, date range, screening rules, and reproducible counts. Those materials were not included with the project source, so this report does not invent publication totals or growth percentages. Instead, this section maps the design to the documented technology areas that directly support the implementation: component-based interface development, REST service construction, browser content scripts, and generative text APIs.

Area reviewed	Relevance to this project	Design implication
Component-based UI	React state and event handling underpin the input, loading, result, and error views [1].	Keep user input and generated output visible and editable.
Backend service	Spring Boot supports the server-side application and web endpoint structure [2].	Keep API key use and model integration on the server.
Browser integration	Manifest V3 defines extension metadata and permissions; content scripts interact with matching pages [3].	Limit host access and expect Gmail DOM changes.
Generative API	Gemini provides text generation through a request/response API [4].	Validate requests and handle model errors and output uncertainty.
UI component system	Material UI supplies React form and feedback components [5].	Use accessible labels and clear generation status.

This thematic review informs implementation choices but is not a systematic literature review. Future academic evaluation should define database searches and screening criteria, then compare the system with baseline email drafting using task completion time, user-rated usefulness, correction rate, and privacy comprehension.

2.4 Review Summary

The background review supports a small, server-mediated assistant with a human review step. React provides a state-driven interface; Spring Boot hosts the API; the Chrome extension connects that API to Gmail; and Gemini supplies candidate language. These technologies solve different parts of the flow and do not by themselves guarantee correctness, privacy, or availability. The system's quality depends on

validation at every boundary: user input, API request, external model call, returned response, and insertion into a draft.

2.5 Problem Definition

Users spend avoidable effort composing routine email replies and may need to switch between their inbox and a separate drafting tool. The project problem is to provide a convenient mechanism that uses the source message and a requested tone to produce a reviewable reply draft, while preserving the user's decision to edit, copy, insert, or discard the result.

The engineering problem is to do so reliably and safely across a browser client, a backend service, and an external model API. The solution must handle empty or malformed input, service failures, response variations, secret management, cross-origin or extension access, and the privacy implications of transmitting email text.

2.6 Goals / Objectives

- Provide a simple input path for email text and optional tone.
- Generate a reply through a server-side model integration without exposing the API secret in the client.
- Make the result easy to review and copy, and insert it into Gmail where the extension is used.
- Communicate request progress and recoverable errors in the user interface.
- Define a testable API contract and validation cases.
- Identify the changes needed for privacy, security, accessibility, and production deployment.

2.7 Why Use These Technologies Over Other Options

Technology	Reason for selection in this project	Trade-off
React	Fits an interactive form and supports local state for input, loading, errors, and result [1].	Requires a JavaScript build/runtime and disciplined component design.
Material UI	Provides ready-made form controls and layout components already used by the interface [5].	Adds dependency weight and requires accessibility review.
Spring Boot / Java	Provides a conventional REST service and server-side configuration for the model call [2].	The current service blocks on the external response and has limited failure handling.
Chrome Manifest V3	Allows an extension content script to augment Gmail pages under declared host access [3].	Gmail markup is not a stable public integration API.
Gemini API	Supplies text generation without hosting a language model locally [4].	External availability, cost, quotas, policy, and data handling apply.

The selected stack is visible in the project manifests and source files. The selection is appropriate for a prototype that demonstrates client-to-server-to-model integration; it does not establish that these technologies are the only or optimal choices for every deployment.

Selection summary. The stack is therefore justified by the prototype's integration goals: React supports the interactive form, Spring Boot provides a server boundary for provider access, and the extension places

a drafting action near Gmail. The trade-offs are dependency management, external-service reliance, browser selector fragility, and additional deployment controls. These choices are appropriate for demonstrating the workflow, while the recommendations in later chapters define the work needed before real-message use.

CHAPTER 3

DESIGN AND PROCESS

3.1 Evaluation and Selection of Specifications / Features

The core specification is intentionally narrow: accept a message and tone, generate one reply, and return it for user review. The feature set below separates code-backed behavior from requirements that remain incomplete or unverified.

Feature	Implementation evidence	Status
Email text entry	React multiline TextField and Gmail DOM extraction	Implemented in source
Tone selection	React offers Professional, Casual, Friendly; extension sends professional	Partial: client behaviors differ
Prompt construction	Service adds a reply instruction, optional tone, and original message	Implemented; tone wording may conflict
Model request	WebClient posts JSON to configured Gemini URL	Implemented in source; live call not verified
Result handling	React result panel and copy; extension inserts into compose editor	Implemented in source; Gmail behavior not verified
Validation and resilience	Web form rejects blank text; backend parser assumes response fields	Partial; server-side safeguards absent
Privacy/security	API key supplied through server configuration; no storage code observed	Partial; transport and access controls need hardening

3.2 Design Constraints

- External model dependency: generation requires network access, a valid credential, an available model endpoint, and sufficient quota.
- Sensitive input: email text can contain personal or confidential information; the service forwards it to an external provider and the current code has no consent or redaction step.
- Credential handling: the service constructs the API URL with a key query parameter. Production deployment should follow current provider guidance and avoid exposing credentials in logs, source control, browser code, or URLs.
- Local endpoint: the React app and extension point to http://localhost:8080, which is suitable only for a local development setup and does not provide encrypted transport.

- Browser page dependence: Gmail class names and editor structure can change; the selectors need monitoring and resilient lookup behavior.
- Prompt consistency: the base prompt asks for a professional reply even when a different tone is selected, which can produce conflicting instructions.
- Response variability: generated text can vary, be empty, or arrive in an unexpected response shape; the parser and UI should handle these cases explicitly.
- No persistent account or history subsystem: the supplied source has no user authentication, stored drafts, analytics, or audit history.

3.3 Design Flow

The user workflow begins with message capture and ends when the user reviews and chooses how to use the draft. The generation request travels from a client to the backend and then to Gemini; the returned text follows the reverse path. Figure 4 summarizes this workflow. Figure 5 separates the client, controller, service, and external model responsibilities.

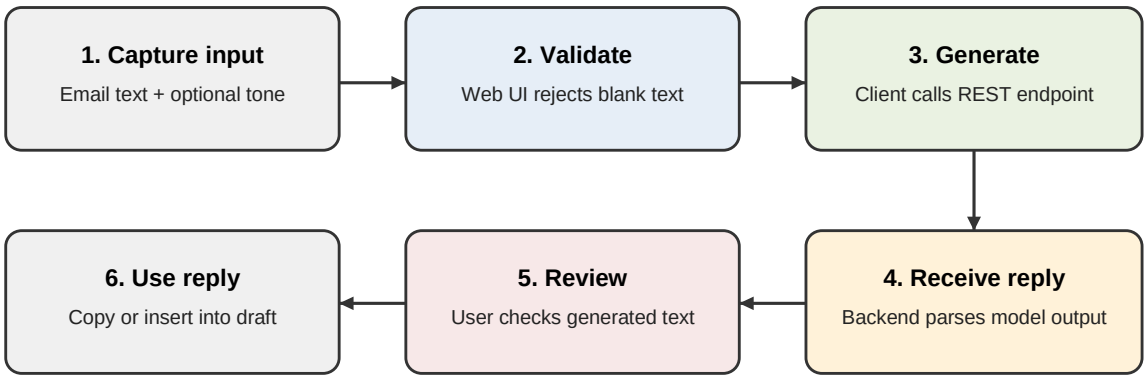


Fig. 4. User workflow for generating a reply

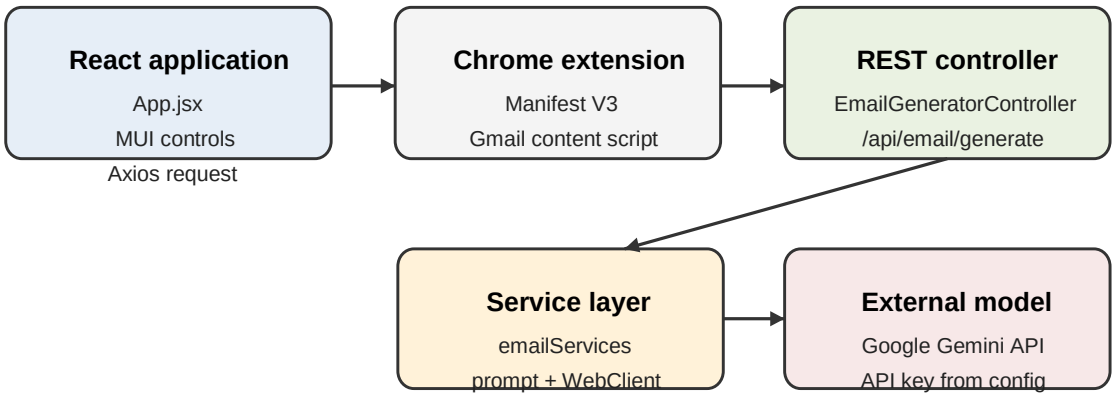


Fig. 5. Software component view

3.4 Creation of SmartMail Assistant

3.4.1 React interface

The React application stores emailContent, tone, generatedReply, loading, and error in component state. The user enters text in a multiline field and can select one of three named tones or leave tone unset. The Generate Reply button is disabled while the text is blank or a request is in progress. On submission, Axios posts JSON to http://localhost:8080/api/email/generate. On success, the returned string is displayed in a result panel; the Copy to Clipboard control uses the browser clipboard API. On failure, a generic retry message appears. The source uses Material UI components for layout, text entry, selection, button, and progress feedback.

The form-level empty-input check improves usability, but equivalent server-side validation is necessary because clients can be bypassed. The current interface also treats successful response data as text or serializes it if it is not a string; a documented response schema would reduce ambiguity.

3.4.2 Spring Boot API

EmailGeneratorController is mapped under /api/email and accepts POST /generate. It binds a JSON EmailRequest containing emailContent and tone, delegates to emailServices.generateReply, and returns the resulting string in an HTTP 200 response. Cross-origin access is currently configured for all origins. The service uses WebClient to submit a JSON contents/parts/text payload to the configured Gemini URL and then parses candidates[0].content.parts[0].text from the response.

The request contract is compact and easy to integrate, but the current controller does not validate null or oversized content, define a typed response object, or translate upstream failures into suitable HTTP statuses. The service uses block() to wait for the model result. Error handling catches parsing exceptions and returns an error string, which makes success and failure difficult for the client to distinguish.

3.4.3 Gmail extension

The Manifest V3 manifest grants activeTab and storage permissions, declares localhost and mail.google.com host access, and loads content.js and content.css on Gmail pages. The content script observes DOM mutations, searches for a compose toolbar, adds an AI Reply button, extracts message text, calls the backend with a fixed professional tone, and inserts the returned text into a Gmail editable textbox. This is a direct DOM integration, not a Gmail API integration.

The implementation has selector-loop control-flow issues: getEmailContent and findComposeToolbar return from the first selector iteration when the first candidate is absent, so later fallback selectors are not tried. The observer's handling of a newly opened compose window is also dependent on which nodes appear in the mutation batch. These issues should be corrected and tested against representative Gmail layouts before claiming dependable integration.

3.4.4 Request and response contract

Element	Type	Purpose
emailContent	String	Source email text used as generation context.
tone	String (optional)	Style instruction such as Professional, Casual, or Friendly.
Request	JSON object	POST body sent to /api/email/generate.

Response	Text string	Generated reply returned to the client; no subject line requested.

The code does not currently enforce an allow-list of tones or a maximum email length. A production API should validate both fields, define a stable JSON response schema, and avoid returning raw implementation errors to the user.

3.5 Logo Designing and Naming

The project is named SmartMail Assistant, a descriptive title that communicates the combination of email drafting and AI assistance. No project-specific logo or brand asset was found in the supplied source files. The report therefore retains the Chandigarh University mark on the cover, as requested, and does not invent a product logo or claim that logo-design work has been completed.

3.6 Implementation Plan / Methodology

The implementation can be understood as an iterative prototype process: establish the backend endpoint and model call, connect a web client, add Gmail page integration, inspect behavior, and refine the response and error paths. The files show the first three activities in code, but no issue tracker, dated plan, deployment record, or end-to-end test evidence was included. Accordingly, the phases below are presented as a repeatable method for completing and validating the system.

Step	Method	Deliverable
1. Define	Record functional and non-functional requirements; agree what email content may be sent to a model.	Requirements and data-flow note
2. Design	Specify the endpoint, payload, response schema, failure states, and trust boundaries.	API contract and component diagram
3. Implement	Build client, API controller, model service, and Gmail content script in small increments.	Versioned code and configuration
4. Validate	Run unit, API, UI, extension, security, and usability checks on test messages.	Test log with evidence and defects
5. Release	Configure HTTPS, secret storage, access controls, monitoring, and user notice.	Reviewed deployment package

3.7 Design decisions and implementation boundary

A shared backend keeps provider communication in one service and avoids placing the Gemini credential in ordinary client code. The trade-off is that both the standalone page and the Gmail action depend on the API being reachable and on the service translating provider outcomes consistently. The web client is a separate Vite application; the Gmail integration is a Manifest V3 content script that operates on rendered page elements. This separation explains why the two paths offer different tone controls and have different failure modes. The current implementation is best treated as a local prototype: its endpoints, CORS policy, credential transport, request validation, and Gmail selectors need further work before broader deployment. The report describes source behavior separately from these recommended changes. Component boundaries should guide verification: prompt tests belong at the backend service, visual and clipboard checks belong in the React client, and selector and editor checks require a browser. A passing result in one layer should not be treated as proof for the others. Keep each check linked to the reviewed source revision. The deployment package should also identify which API address the extension expects and which origins the service accepts. Review those values after every environment change so that a development configuration is not accidentally mistaken for a production one.

CHAPTER 4

RESULT ANALYSIS AND VALIDATION

4.1 Implementation of Solution

Source inspection confirms a functional path has been implemented at the code level: a client builds an email/tone request, the controller forwards it to a service, the service calls the configured Gemini endpoint, and reply text is returned. The React path displays and copies the result. The Gmail script attempts to add a compose action and insert the result. These statements describe source behavior; no live Gemini response, browser session, benchmark, or user study was supplied as evidence.

The resulting prototype demonstrates the intended architecture but has incomplete validation and deployment support. Table 4.1 summarizes feature coverage and evidence limits.

Area	Observed result in source	Evidence boundary
Web input	React form captures content and optional tone; blank input is rejected.	No rendered or browser execution evidence supplied.
Backend route	POST /api/email/generate binds EmailRequest and returns text.	No running API trace supplied.
Gemini integration	Service builds a prompt and extracts candidate text.	No credential or live provider result supplied.
Gmail action	Content script attempts button injection and draft insertion.	Gmail DOM behavior not exercised in evidence.
Error handling	Web UI shows a generic error; parser returns an error string.	Failure status contract is not defined.
Test coverage	A Spring application-context test class exists in the repository.	No test execution result is reported here.

4.1.1 Report Preparation

The report was prepared from the supplied repository, including the README, React component and package metadata, Spring Boot controller and service, application properties, extension manifest, and Gmail content script. The PDF format provided the chapter sequence and visual pattern. No screenshots, user records, production logs, measured response times, or formal test reports were present in those materials. The report therefore uses architecture diagrams and explicitly identified implementation summaries in place of unsupported screenshots or quantitative results.

4.1.2 Project Management and Communication

The repository contains separate directories for the React application, backend, and browser extension, which makes the principal technical boundaries identifiable. The README documents a local setup concept and API example, but some of its claims and paths do not match the current source layout. For example, it describes React/MUI as the extension UI while the manifest points to a standalone content

script and the React app is a separate Vite project. Project handover should reconcile the README with actual directories, configuration, tone behavior, and installation steps.

No meeting notes, work assignments, milestone records, or version-control history analysis were supplied as project evidence. This section does not attribute team actions or dates that cannot be verified.

4.1.3 Testing / Characterization / Interpretation / Data Validation

The following matrix is a proposed validation plan derived from the implementation. It records expected behavior and is not a claim that tests have been run. Each case should be executed with a non-sensitive test message after the endpoint and extension are configured.

ID	Scenario	Expected result	Status in this report
V1	Valid email with Professional tone	API returns a non-empty reply draft.	Not executed
V2	Valid email with Casual/Friendly tone	Reply follows selected tone without conflicting default instruction.	Not executed
V3	Blank or whitespace-only request to API	Client/server rejects it with a clear validation response.	Not executed; server validation absent
V4	Gemini timeout, quota error, or invalid key	Client receives a meaningful failure status; secret is not exposed.	Not executed; upstream mapping incomplete
V5	Response lacks candidates or text part	Service returns a controlled error, not an uncaught exception.	Not executed; parser assumes shape
V6	Gmail page lacks first toolbar selector	Fallback selectors are tried and no duplicate button is created.	Not executed; loop returns early
V7	Long email input	Length limit and user feedback are applied before model call.	Not executed; limit absent
V8	Check browser network and server logs	No API key is exposed; transport is encrypted in deployment.	Not executed; local HTTP configured

Interpretation: implementation coverage is strongest for the happy-path sequence and the web interface's local user feedback. Validation coverage, privacy controls, upstream failure handling, Gmail selector resilience, and deployment hardening remain the priority areas. No performance or accuracy metric is reported because no measurements were supplied or collected.

4.1.4 Acceptance criteria and evidence handling

A validation result should identify the source revision, environment, test case, observed behavior, and evidence reference. Mark each case as passed, failed, blocked, or not run; do not infer a pass from source inspection or from a screenshot. Use synthetic or consented messages, redact account details, and avoid storing email text in logs or worksheets. Acceptance for a controlled demonstration should cover a valid request, invalid input, provider failure, client recovery, and the visible Gmail action. Any claim of successful insertion requires a browser observation that confirms the intended compose editor changed while existing draft text remained intact and the Send action was not activated. Production acceptance also requires security, privacy, and accessibility review.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

SmartMail Assistant demonstrates a basic AI-assisted email reply workflow using a React and Material UI front end, a Spring Boot REST API, a Gemini model call, and a Chrome content script for Gmail. The design keeps generation behind a backend service and returns a draft for the user to review. The project therefore establishes a clear prototype architecture for reducing repetitive reply-writing effort.

The supplied implementation does not yet establish reliable production behavior. The browser and web clients have differing tone capabilities; Gmail integration relies on brittle selectors; the backend lacks robust request validation and upstream error mapping; the model response parser assumes a fixed shape; and the configured local HTTP endpoint is not suitable for deployment. Email privacy, credential handling, accessibility, and testing also need explicit treatment. These findings define the next engineering steps rather than diminishing the prototype's value as an integration demonstration.

5.2 Future Work

5.2.1 Improve prompt and response handling

Build the prompt from a neutral instruction plus exactly one validated tone choice. Request concise output without an invented subject line, and consider a structured response containing reply text and safe status metadata. Parse missing or blocked model output explicitly, set timeouts, and translate provider failures into stable HTTP error responses.

5.2.2 Strengthen privacy and security

Use HTTPS for all non-local traffic, keep API credentials in a secret manager or environment configuration, and follow the model provider's current authentication guidance. Add input-size limits, rate limiting, abuse controls, origin restrictions, and a user-facing explanation of where message content is processed. Avoid logging message content or secrets. Review the data-retention and processing terms of the selected model service before deployment.

5.2.3 Make the Gmail integration more reliable

Correct selector iteration so all fallbacks are tried, scope message and compose elements to the active conversation, prevent duplicate controls, and handle Gmail navigation changes. Prefer supported extension and mail APIs where the required functionality permits. Add compatibility checks and a graceful fallback that lets the user copy the response if insertion fails.

5.2.4 Expand validation

Add service-level tests for prompt construction and response parsing, controller tests for validation and status codes, and integration tests using a mocked model response. Exercise the extension against test Gmail layouts, including missing toolbar elements and multiple compose windows. Conduct usability checks for tone appropriateness, edit effort, and user understanding of the model's limitations. Report measured results with sample size and test conditions.

5.2.5 Complete deployment and accessibility work

Reconcile the README with the actual directory structure, provide a reproducible setup, configure a production API URL and deployment profile, and monitor availability without collecting message content. Review keyboard access, focus order, accessible labels, progress announcements, and error descriptions in both client paths. Future extensions may include user-defined tone guidance, language selection, and draft refinement, provided they do not obscure the user's review and control.

5.3 Prioritized implementation roadmap

The next development cycle should address the current prototype's highest-impact gaps in a sequence that can be verified. The work below is derived from the supplied React, Spring Boot, and extension code. It does not imply that these changes have already been implemented.

Priority	Project work	Acceptance evidence
1. API reliability	Validate emailContent and tone on the server; define response and error shapes; handle empty provider output and set a request timeout.	Direct API cases reject invalid input; clients recover from provider errors.
2. Privacy and deployment	Keep credentials in protected backend configuration; use HTTPS and restrictive CORS; redact logs; explain provider processing to users.	Secret scan and origin checks pass; configuration and privacy notice are reviewed.
3. Gmail integration	Review selector fallbacks, scope each action to its compose window, preserve existing draft text, and support keyboard activation.	Test new, reply, and multiple compose windows; verify Send is never triggered.
4. Evaluation	Run synthetic end-to-end cases and record revision, environment, observed result, and evidence. Measure user effort only with a defined study.	Completed case log; no unverified reliability or productivity claims.

Release decision

The prototype is ready for a broader demonstration only when the request path, failure behavior, secret handling, and Gmail draft behavior have been checked in the target environment. A local success should not be treated as evidence of production security or provider availability. Keep a known source revision, use fictional or consented messages during validation, and retain screenshots and logs only after removing account details and credentials.

Validation exit checks

- API: reject blank, oversized, and unsupported-tone requests before contacting Gemini; return a documented failure for provider errors.
- Web client: verify loading resets on every outcome, errors clear at the right time, and copied text preserves paragraph breaks.
- Gmail: test new, reply, and multiple compose windows; confirm extraction is scoped correctly and existing draft text is preserved.
- Safety: confirm the extension never activates Send and the generated text remains an editable draft for user review.
- Privacy: verify credentials stay out of browser assets and logs, origins are restricted, and the data notice matches the deployment.

- Accessibility: check keyboard operation, visible focus, control labels, loading announcement, error message, and page zoom.

REFERENCES

[1] React Team, *Quick Start*, React documentation. Available: https://react.dev/learn (accessed 29 October 2026).

[2] VMware, Inc., *Spring Boot Reference Documentation*. Available: https://docs.spring.io/spring-boot/reference/ (accessed 29 October 2026).

[3] Chrome for Developers, *Manifest file format*. Available: https://developer.chrome.com/docs/extensions/reference/manifest (accessed 29 October 2026).

[4] Google AI for Developers, *Gemini API: Generating content*. Available: https://ai.google.dev/api/generate-content (accessed 29 October 2026).

[5] MUI, *Material UI documentation*. Available: https://mui.com/material-ui/getting-started/ (accessed 29 October 2026).

[6] R. Soni, *SmartMail Assistant project source repository*, supplied project files: README, React interface, Spring Boot backend, and Chrome extension. Reviewed 29 October 2026.

PLAGIARISM REPORT

Attach the institution-issued similarity report here after the report has been processed through the approved Chandigarh University system.

APPENDIX A - SOURCE CODE AND SYSTEM WALKTHROUGH

A.1 Repository map and system boundary

The supplied project is divided into three executable areas: a Vite and React web interface, a Spring Boot service, and a Chrome extension content script. The README describes the intended end-to-end feature, but its illustrative folder names do not exactly match the current repository. For this report, the checked-in files are treated as the source of truth when code behavior differs from prose documentation.

Implementation notes

- email-writer-react/src/App.jsx contains the visible form, asynchronous request, loading state, output panel, and copy action.
- EmailWriter-backend/src/main/java contains the request DTO, REST controller, application class, and Gemini service.
- EmailWriterExtension/manifest.json and content.js describe the Gmail page integration; no bundled React extension popup is present in the manifest.

Review and interpretation

Keep the web app and extension diagrams as separate client paths that converge on one backend endpoint. Do not describe absent account, billing, storage, or mailbox-send functionality as implemented.

A.2 React entry point and component responsibilities

The React entry point imports the global stylesheet and renders App inside StrictMode. The App component is a single-page interaction surface rather than a multi-route product. Its controls are composed from Material UI, while Axios is used for the HTTP request. This arrangement makes the prototype easy to inspect, but it places most presentation, input handling, request orchestration, and output logic in one component.

Implementation notes

- main.jsx creates the React root using the #root element from index.html.
- App.jsx owns five state values: source text, selected tone, generated text, loading status, and user-facing error.
- The supplied package uses React 19, Material UI 7, Axios 1, and Vite 7; these versions are read from the project manifest.

Review and interpretation

A future refactor could extract a reply form, tone selector, output panel, and request hook. Such arefactor should preserve the currently visible behavior and should be documented as future work unless completed.

System boundary. The three code areas should be read as a connected prototype rather than as a single deployable package. The React page and Gmail content script have different execution contexts, but both depend on the same backend route. That shared route is the strongest architectural seam for validation and monitoring. It also creates a common failure point: unavailable API configuration or provider access affects both clients. Keep the report's diagrams and claims aligned with these boundaries, and identify the exact source revision whenever screenshots or setup notes are refreshed.

A.3 Input state and tone selection

The original message is held in component state and displayed in a multiline text field. The tone control offers an empty option plus Professional, Casual, and Friendly. The user may submit with no explicit

tone. The project has no server-side allow-list, so the request DTO currently accepts whatever string a client supplies. The extension sends a fixed tone value rather than presenting this selector.

Implementation notes

- The web interface trims input only for its blank check; it sends the original entered text in the JSON payload.
- Changing the tone updates the React state and is included in the next request.
- The extension's fixed professional setting means the available tone behavior is not consistent across the two clients.

Review and interpretation

A clear contract should define accepted tone identifiers, their exact wording, and the behavior formissing or unrecognized values. Server validation should be authoritative even when the UI constrains its options.

A.4 Submission state and request lifecycle

When Generate Reply is activated, the interface clears a previous error and sets loading to true. Axios posts emailContent and tone to the local API route. While awaiting the response, the button is disabled and a progress indicator is rendered. A successful response is assigned to generatedReply. A rejected request triggers a generic failure message. A finally block restores the loading state regardless of success or failure.

Implementation notes

- The source prevents a whitespace-only request from reaching the API through the web form.
- The user sees a spinner but no cancellation action or request timeout configuration in App.jsx.
- The catch block discards diagnostic details and presents a short retry message.

Review and interpretation

Keeping detailed diagnostic output in developer logs while showing safe, useful error categories to the user would improve support. Avoid placing message text or credentials in logs.

State and request lifecycle. The client lifecycle begins before the request: input is entered, tone is chosen or omitted, and blank content is screened. The request then becomes an asynchronous operation with a loading state and a result or error. The current component stores those values independently, so transition rules matter: old output should not be mistaken for a new response, and loading must end after every failure. The extension has a similar lifecycle but lacks the web form's selectable tone. Treat each client as a separate user experience that shares only the service contract. **Verification detail.** A practical review should also check the generated text after an input edit, after a second submission, and after a request that returns no usable content. These transitions reveal stale-state defects that a single successful example will not expose. **Release consideration.** A useful acceptance example is a short fictional scheduling message: the output should retain the stated time, avoid inventing a meeting link, and remain editable. Repeat with an empty tone to make the default behavior observable.

A.5 Generated reply display and clipboard action

A non-empty generatedReply renders a bordered result panel. The reply is shown with line breaks preserved, which helps retain paragraph formatting returned by the model. The Copy to Clipboard button uses navigator.clipboard.writeText. The source does not show a copy confirmation state, fallback for clipboard permission failure, or an edit field separate from the displayed response.

Implementation notes

- The panel distinguishes the generated text with a heading.
- Clipboard writing is initiated only after the user activates the copy control.
- The application does not submit or send the email on the user's behalf.

Review and interpretation

For a production interface, indicate whether copy succeeded, preserve keyboard focus, and let the user edit the proposed text before copying. Keep the final decision to send with the user.

A.6 REST controller and route

EmailGeneratorController is annotated as a REST controller and uses the base mapping /api/email. Its POST mapping /generate binds a JSON EmailRequest and returns ResponseEntity. The controller delegates the work to emailServices and wraps the returned text in an HTTP 200 response.
Cross-origin requests are currently permitted from any origin by the annotation.

Implementation notes

- The public route used by both clients is POST /api/email/generate.
- The controller does not currently expose a separate health or readiness endpoint in the inspected source.
- There is no controller-level validation or exception-to-status mapping shown.

Review and interpretation

Before external deployment, use an explicit origin policy, validate incoming payloads, and return a documented response model with appropriate HTTP status codes for invalid input and upstream failures.

Request	Current code behavior	Recommended contract
Method and path	POST /api/email/generate	Retain or version intentionally
Body	emailContent and tone	Validate type, length, and accepted tone
Response	Raw string	Stable JSON response object

API responsibility. The route should remain a narrow orchestration boundary: bind the request, validate it, call the generation service, and translate outcomes into a documented response. Keeping provider-specific parsing out of the controller preserves separation of concerns. A typed response would allow clients to distinguish generated text from validation or upstream errors without inspecting message strings. The current raw-string response is compact for a demonstration, but it gives the clients little structured information for recovery or telemetry. **Verification detail.** Keep the API's responsibilities small enough that a future provider can be substituted without changing Gmail-specific selectors or React presentation. This separation also makes it easier to test provider parsing independently from user interaction.

A.7 Request DTO and application configuration

EmailRequest uses Lombok's Data annotation to supply accessors for emailContent and tone. The backend configuration names the application EmailWritter and resolves gemini.api.url and gemini.api.key from environment properties. The URL and key are configuration inputs; no literal credential is present in the checked-in application.properties file. A deployment must still provide valid values at runtime.

Implementation notes

- The DTO does not declare explicit validation annotations in the supplied file.

- The project uses Java 17 in the Maven properties.
- The property placeholders use GEMINI_URL and GEMINI_KEY, so local setup instructions need to explain both values without publishing secrets.

Review and interpretation

Keep local secrets outside version control, fail startup clearly when required configuration is absent, and separate local development settings from production deployment settings.

A.8 Prompt construction

The service constructs an instruction asking for a professional email reply and no subject line. If a non-empty tone is supplied, a second instruction asks Gemini to use that tone. The original email is appended after an Original Email label. This prompt is concise and demonstrates context passing, but the default professional instruction can conflict with a Casual or Friendly selection.

Implementation notes

- The prompt includes the submitted email body as model input.
- The source does not currently label the incoming email as untrusted content or instruct the model to ignore instructions embedded within it.
- The prompt does not explicitly constrain length, factual invention, or use of placeholders for missing facts.

Review and interpretation

Prompt changes should be reviewed with test cases for each supported tone and adversarial email content. The generated result should remain an untrusted draft that the user checks before use.

Prompt and provider boundary. Prompt construction is part of application behavior and should be versioned with the code. Tone wording, message delimiters, and output constraints affect the returned text even when the UI is unchanged. Provider request and parsing code should be isolated behind a service so fixtures can exercise missing candidates, empty parts, malformed JSON, and provider error payloads. Avoid logging the assembled prompt: it contains the user's email text and may reveal private information. **Verification detail.** A response object could carry a stable error code and safe user message while leaving provider diagnostics private. The schema should be chosen before adding more clients, then documented with representative success and failure examples. **Release consideration.** Do not expose provider-specific fields in the client contract unless users need them. A small adapter in the service can normalize the upstream result and keep changes to the external schema from leaking into React and Gmail code.

A.9 External request and response extraction

WebClient sends a JSON payload with a contents array, parts array, and a text field containing the constructed prompt. The response is read as a string and parsed with Jackson. The service selects the first candidate and first text part. Parsing exceptions are caught and returned as an error string. Transport exceptions and non-success HTTP responses are not normalized into a typed error model in the supplied implementation.

Implementation notes

- The API key is currently appended to the configured URL as a query parameter.
- The parser assumes at least one candidate and one part with text.
- The service blocks on the reactive publisher using block(), so the request thread waits for the provider response.

Review and interpretation

Add timeout and retry policy only after considering duplicate-generation costs. Handle missing candidate data, refusal responses, quota failures, and network errors with explicit outcomes.

A.10 Chrome extension manifest and content script

The extension declares Manifest V3, activeTab and storage permissions, localhost and Gmail host access, and a content script matched to mail.google.com. The script observes the page for compose-related mutations, searches for a toolbar, creates a button, reads visible message content, calls the local API, and inserts returned text into an editable compose area. It uses Gmail's DOM directly, not a supported Gmail API workflow.

Implementation notes

- The action manifest provides a title but does not declare a popup page.
- The content script includes fallback CSS selectors, but two helper loops return on their first iteration when the first selector misses.
- The extension sends the fixed tone string professional and uses the same localhost endpoint as the web UI.

Review and interpretation

Test selector behavior with fresh Gmail page structures and multiple open compose windows. The manifest should request only the permissions and origins needed by the release configuration.

Browser integration boundary. The extension reaches into Gmail's rendered page rather than using a stable mail API. Its selectors, observer timing, and choice of compose editor therefore need explicit regression checks. The current helper-loop behavior can stop before trying later fallbacks, and the action must be scoped to the intended conversation and draft. A safe workflow generates editable text only; it should neither activate Send nor silently overwrite existing draft content. **Verification detail.** For each provider response shape, use a fixture that is independent of network credentials. The service should distinguish an empty but valid response from malformed data and report both through predictable client outcomes. **Release consideration.** The extension's local endpoint and fixed tone should be treated as configuration differences, not as evidence of a separate model. If the service response changes, test the extension independently because insertion and React rendering consume the response in different ways.

APPENDIX B - OPERATIONS AND USER GUIDE

B.1 Prerequisites and workspace layout

The README lists Node.js 18 or newer, npm, Java 17 or newer, Maven, and a Google AI project with Gemini API access as prerequisites. The checked-in frontend and backend are in separate directories, and the extension source is in its own folder. A fresh setup should verify runtime versions and install dependencies within the corresponding project before attempting a demonstration.

Implementation notes

- Frontend directory: email-writer-react.
- Backend directory: EmailWriter-backend.
- Extension source directory: EmailWriterExtension; the supplied manifest is not a built React extension package.

Review and interpretation

The commands in the README should be reconciled with the actual repository paths before the report is used as a handover manual. Never commit generated API credentials.

B.2 Configure the Gemini service

The backend reads the Gemini URL and API key from GEMINI_URL and GEMINI_KEY placeholders. The README gives an example endpoint but the project properties are the runtime source of configuration. A developer should set these values in the local run environment or a protected configuration store. The exact model name and endpoint must match the provider account and current API documentation.

Implementation notes

- Set the URL and credential only in the backend environment.
- Do not place the key in React, the extension manifest, content.js, screenshots, or a report appendix.
- Confirm that the configured URL is reachable before testing user-visible generation.

Review and interpretation

The current service appends the key to the URL. A production update should align with current provider authentication guidance and ensure URLs containing secrets are not retained in logs or monitoring.

Reproducible setup. Setup instructions should identify the actual directories and distinguish frontend, backend, and extension startup steps. The backend requires provider configuration, whereas the browser clients require a reachable API address. A developer should be able to start the API first, confirm configuration without displaying the key, then start the web client or load the extension. Keep the example commands local and avoid implying a public deployment. Record runtime versions because Node, Java, Maven, browser behavior, and provider endpoints can affect reproduction. **Verification detail.** A reproducibility record should include the selected client path, the API host category, runtime versions, and whether the provider was mocked. Keep credentials out of notes, screenshots, and terminal captures used for submission. **Release consideration.** A handover should state which terminal runs each service and how to stop it cleanly. Document expected local ports and a non-sensitive smoke request, but do not include a working provider key in the report or screenshots.

B.3 Start the Spring Boot backend

From the EmailWriter-backend directory, the Maven wrapper can be used to launch the service in a development environment after dependencies and configuration are available. The project's controller is expected to listen on port 8080 according to the README's local setup. Startup should be considered successful only when the process reports that it is listening and the required Gemini properties have been resolved.

Implementation notes

- Windows wrapper: mvnw.cmd spring-boot:run.
- Check startup logs for missing placeholders or dependency errors.
- Keep the backend terminal open while using either client.

Review and interpretation

A production deployment should use a managed runtime, externalized secrets, HTTPS termination, structured health checks, and resource limits. The README currently describes a local demonstration, not a production operations plan.

B.4 Start the React web interface

The frontend package defines Vite scripts for development, build, lint, and preview. For a local demonstration, install the package dependencies and start the development server from the email-writer-

react directory. The current React source sends requests directly to http://localhost:8080, so an API running elsewhere will not be reached without updating the application configuration or client code.

Implementation notes

- Use npm install in the frontend directory when dependencies are not present.
- Use npm run dev to open the local Vite interface.
- Use npm run build for a static production build; deployment routing and API URL handling still need configuration.

Review and interpretation

The endpoint should be configurable by environment or build-time settings rather than hard-coded, and cross-origin policy should match the deployment origin.

Configuration discipline. The provider key belongs only in backend-controlled configuration. The browser code should receive an API base URL appropriate to its environment, never the provider credential. Before a demonstration, verify that the client and backend point to the same host and that the backend can reach the configured provider. Use synthetic content during this check. The present fixed localhost addresses are useful for a local prototype but must be replaced for any remote environment, together with HTTPS and a restrictive origin policy. **Verification detail.** Configuration validation should fail clearly when a required provider value is missing, but startup and error messages must not print the value itself. Separate development examples from deployed configuration and review every client bundle before publishing it. **Release consideration.** Configuration should have a documented source of truth and a startup check for required values. Keep environment names consistent with the application properties and README, and verify that error messages describe missing configuration without echoing secrets.

B.5 Web interface: prepare the message

Open the Email Reply Generator page and paste a representative source email into Original Email Content. Use a fictional or approved test message rather than confidential personal or company correspondence. The project sends the entered text to its backend, which forwards it to Gemini. The user should understand this path before entering real email content.

Implementation notes

- Include enough context to make a reply possible, but omit unnecessary personal data.
- Do not include passwords, access codes, payment information, or private attachments in a test.
- The current form checks that text is not blank before enabling Generate Reply.

Review and interpretation

The application does not currently show a data-processing notice or redaction step; those should be included before real-world use.

B.6 Web interface: choose a tone and generate

Select Professional, Casual, Friendly, or None in the optional tone control, then activate Generate Reply. The button enters a loading state while Axios waits for the API response. When the service succeeds, the returned string appears below the input. The request is synchronous from the user's perspective, and the current UI has no stop or cancel action.

Implementation notes

- Compare the generated wording with the source email and verify names, dates, amounts, commitments, and factual details.

- Edit the response before using it whenever the model has omitted context or added unsupported claims.
- A tone selection cannot guarantee that every sentence follows the requested style.

Review and interpretation

The default prompt's Professional instruction can conflict with other choices. A verified release should harmonize the prompt before claiming all tone options work as expected.

Responsible message preparation. Email content can contain details unrelated to the requested reply. Users should provide the minimum context needed, remove credentials and confidential attachments, and understand that the current path forwards the text to an external model provider. This is a data-handling concern, not merely an input-form concern. A deployment should show a notice before real use and should describe the actual provider, retention, and logging practices rather than general assurances. **Verification detail.** A concise user notice should explain what text is sent, the role of the external model, and the user's responsibility to review a reply. This information should be available before a real message is submitted, not only in a technical appendix. **Release consideration.** A message-preparation example should use fictional details and demonstrate what the user chooses to omit. Include no account address or real correspondence in examples; a realistic but synthetic message is sufficient to exercise reply drafting.

B.7 Use the Chrome extension

The extension content script is designed to run on Gmail pages, add an AI Reply action to a compose toolbar, obtain visible email text, call the local backend, and insert the response into the draft editor. The manifest grants access to Gmail and the local API. This integration depends on Gmail page markup and must be manually verified in a test mailbox after loading the extension unpacked.

Implementation notes

- The manifest declares Manifest V3 and matches mail.google.com pages.
- The current source uses fixed local HTTP and a fixed professional tone.
- Review the draft before sending; the extension does not send mail automatically.

Review and interpretation

If injection or insertion fails, use the standalone interface or copy workflow where available and record the browser layout and error for debugging.

B.8 Troubleshooting and support notes

A failed generation can originate in the web interface, the local API, model configuration, provider availability, or browser network policy. The current interface intentionally hides detailed exceptions from the user, so diagnosis should be performed with local developer tools and backend logs while avoiding message text or secrets. The extension also depends on selectors that can stop matching after a Gmail UI change.

Implementation notes

- If the button remains disabled, verify that the input contains non-whitespace characters.
- If the request fails, verify backend availability and environment configuration without copying credentials into logs.
- If Gmail controls are not found, reload a test page only after confirming no unsaved draft is at risk.

Review and interpretation

A support runbook should identify the deployed version, endpoint health, provider status, and client browser version, while minimizing collection of email content.

Failure recovery. The interface should make a failed request recoverable: preserve the user's source text, restore the action button, clear transient loading indicators, and show a concise reason where the service can safely provide one. Provider diagnostics belong in protected backend logs and should be redacted. A retry should be a deliberate user action because a second provider call can incur cost or return different wording. The same recovery expectations apply to Gmail insertion, where a failure must leave the existing draft untouched. **Verification detail.** The extension's failure path should preserve the compose contents and allow the user to continue manually. If selector discovery fails, provide a clear non-destructive status rather than guessing another editor or conversation. **Release consideration.** Troubleshooting should first identify whether the failure occurred before the API call, during provider access, or during editor selection. Safe status categories can guide the user without revealing raw exception text or encouraging repeated requests that may incur cost.

APPENDIX C - APPLICATION SCREENSHOTS

C.1 Web application - initial interface

The supplied screenshot shows the SmartMail Assistant web form before message entry. The visible controls are the email content field, optional tone selector, and Generate Reply button. The disabled button reflects the empty form state in the supplied capture.

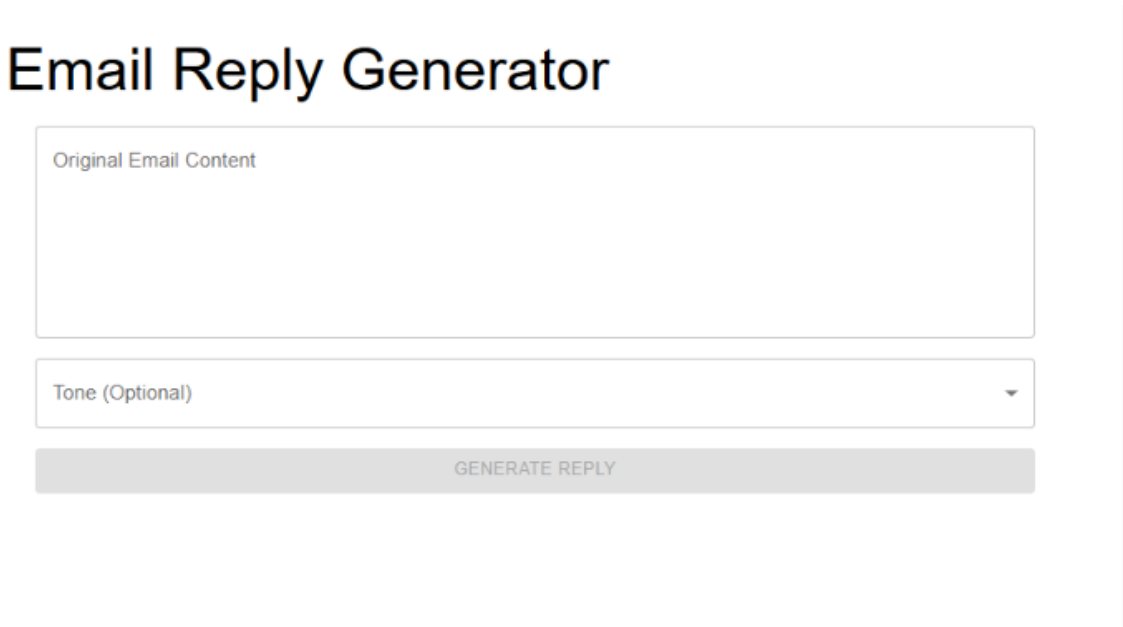


Fig. C.1. SmartMail Assistant web interface in its initial state

Observed state: the email field is empty, no tone is selected, and Generate Reply is disabled. This capture documents the initial interface and blank-input state; it does not demonstrate a backend request.

Working sequence represented by this view

- Enter a concise incoming message and select Professional, Casual, Friendly, or no explicit tone.
- The web form prevents a blank submission; Generate Reply starts the request and shows a loading state.
- React posts emailContent and tone to the Spring Boot route, which builds a prompt and calls the configured Gemini endpoint.
- The returned text appears in the result panel and can be copied. The user checks facts, names, dates, and commitments before use.

- If the request fails, the interface shows a generic retry message; no email is sent automatically.

Verification boundary: this supplied image documents the empty initial form, not a live response. A working demonstration should use synthetic text, capture the loading and returned-reply states, and confirm that copied output matches the result panel. Until that evidence is recorded, the report describes the request path from source inspection rather than a provider run.

C.2 Gmail compose view - AI Reply action

The supplied screenshot shows a Gmail compose area with the AI Reply control beside the standard compose actions. It documents the extension action in the draft interface; it does not show a generated result or confirm that a live backend request completed.

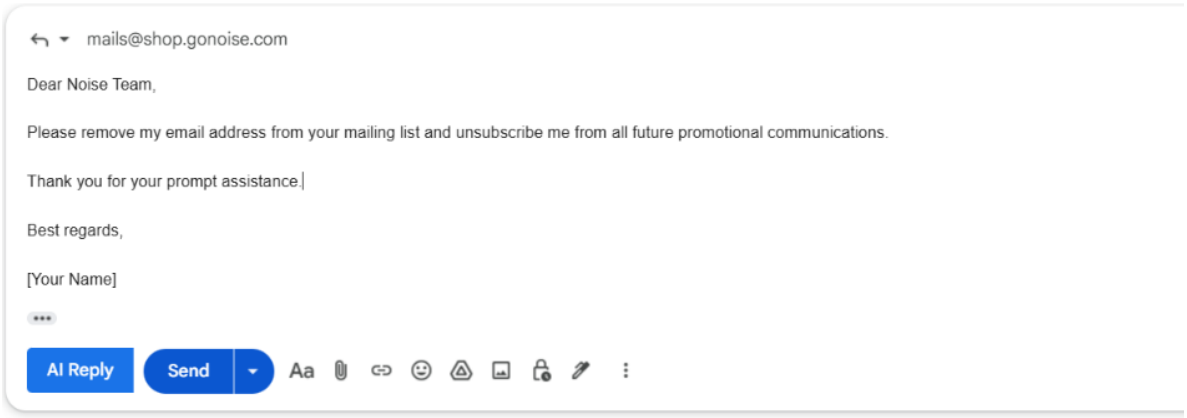


Fig. C.2. Gmail compose view with the AI Reply extension action

Observed state: a Gmail compose area shows the AI Reply control beside standard compose actions. The capture documents control placement only; it does not show a generated reply or confirm draft insertion.

Working sequence represented by this view

- Open the relevant Gmail conversation and create or open its compose surface.
- The content script detects the compose toolbar and adds the AI Reply control shown in the screenshot.
- Activating the control reads visible message text and sends it with the fixed professional tone to the shared local API.
- The script attempts to insert returned text into the editable compose area; the user reviews and sends manually if appropriate.
- This screenshot confirms control placement only; live generation and correct insertion need separate browser evidence.

Verification checklist

- Check a new message, a reply, and multiple compose windows; the control should appear once in the intended surface.
- Compare extracted text with the selected conversation and verify existing draft text is preserved after insertion.
- Force a provider failure and confirm that the action recovers without changing the draft.
- Confirm the extension never activates Send; review and sending remain manual user actions.
- Use synthetic content and capture redacted before-and-after evidence for the same test case.

APPENDIX D - API, DATA HANDLING, AND SECURITY

D.1 Data journey and processing boundaries

The email text begins in a React field or is read from the Gmail page. The client sends it with a tone to the local backend. The backend composes a prompt and sends that prompt to the configured Gemini endpoint. The model response is parsed and returned to the client, where it is rendered or inserted into a draft. This flow crosses the browser, backend, and provider trust boundaries.

Implementation notes

- The repository shows no database writes for message content.
- The absence of storage code is not a proof that no operational logs or provider retention exist.
- The user must review the provider and deployment data policies before using sensitive email.

Review and interpretation

A data-flow diagram for a real deployment should identify collection, transit encryption, processing, logging, retention, access roles, and deletion practices.

D.2 Data minimization and purpose limitation

A reply-generation request needs the content required to draft a response, but not necessarily an entire thread or unrelated signature history. The current client reads a selector's innerText and can therefore capture more context than intended, depending on Gmail's DOM. The standalone UI sends everything pasted by the user. A safer design makes the transmitted scope visible and gives the user a chance to remove irrelevant sections.

Implementation notes

- Limit the selected message to the active conversation content needed for the reply.
- Strip unnecessary quoted history only when doing so does not remove essential context.
- Avoid collecting contact lists, attachments, or mailbox metadata for a text-only reply feature.

Review and interpretation

Minimization should be tested with long threads, quoted replies, signatures, and forwarded messages to ensure the user can understand what leaves the client.

Request and response trust boundary. The browser request is untrusted even when it originates from the project's own interface. A direct caller can bypass client-side checks, send unsupported tones, or submit oversized content. Server-side validation should therefore define accepted fields, limits, and error outcomes. The provider response is also external data: the service must handle absent or empty candidate text and must not expose raw parsing exceptions to clients. These controls turn a happy-path demonstration into a predictable API. **Verification detail.** Validation rules should be applied before downstream work begins, which limits avoidable provider calls and keeps error ownership clear. Test direct API requests because browser-side disabled controls do not protect a server endpoint. **Release consideration.** The controller should distinguish binding errors from service errors and should return the same documented schema for both clients. Verify that a malformed request never reaches prompt construction and that provider errors do not become HTTP 200 success strings.

D.3 Threat model and trust boundaries

The main assets are the provider credential, email text, generated response, user session, backend availability, and extension permissions. Relevant threats include credential disclosure, unauthorized API use, prompt injection embedded in a message, cross-origin abuse, denial of service, accidental

transmission of private content, and malicious or incorrect generated wording. The supplied prototype has no formal threat model artifact.

Implementation notes

- Treat email text as untrusted input even when it is displayed in a trusted mailbox.
- Treat model output as untrusted content and never execute it as HTML or code.
- Constrain who can call the backend and how much work each request can trigger.

Review and interpretation

A security review should rank threats by likelihood and impact and link each mitigation to a test case and deployment control.

D.4 Authentication, authorization, and quotas

No user authentication or authorization layer is present in the controller. With CORS set to all origins, any browser origin able to reach the endpoint may attempt a request, subject to network controls. A publicly accessible service would need a deliberate access model, quota enforcement, and abuse protections so that an exposed route cannot be used to spend provider quota or process unwanted content.

Implementation notes

- Choose whether the service is single-user, organization-restricted, or public before deployment.
- Apply per-user or per-client rate controls and request size limits.
- Protect any user identity token and avoid treating CORS as authentication.

Review and interpretation

Authorization should be applied by the backend and verified with negative tests for missing, invalid, and insufficient credentials.

Data minimization. The feature needs message text to draft a reply, but it does not need unrelated account data, mailbox history, or credentials. Extract only the intended conversation context and avoid storing content in application logs or analytics. The request may include quoted history, signatures, and personal details; the user should be able to inspect what is submitted where practical. A privacy notice must match the real network path and the actual provider terms for the deployment. **Verification detail.** Retention decisions must cover more than an application database. Reverse proxies, service logs, provider request records, screenshots, and browser debugging tools can all retain content unless the deployment is configured deliberately. **Release consideration.** The data map should be reviewed whenever features are added. Accounts, history, analytics, or attachment processing would change the current privacy boundary and require additional storage, access, retention, and user-notice decisions.

D.5 Input validation and resource limits

The current web form rejects a blank message, but a caller can bypass that UI and post directly to the controller. The DTO does not declare length or content validation. Extremely long input can increase provider cost and latency; malformed JSON or missing fields can produce failures that are not converted to a clear client response. Validation is needed before prompt construction.

Implementation notes

- Require non-empty emailContent and enforce a documented maximum size.
- Accept only recognized tone values or normalize unknown values safely.
- Set request timeouts and guard against repeated generation requests.

Review and interpretation

Validation rules should be shared as contract documentation but enforced on the server. Return field-specific messages without echoing private email text.

D.6 API key and credential handling

The checked-in properties use environment placeholders instead of hard-coded key values. The service then appends the key to the configured URL. URL-based secrets can be captured by proxies, diagnostics, browser or server logs, and monitoring systems. The provider's current authentication documentation should guide a secure request header and key scope. The key belongs only in the server environment or secret store.

Implementation notes

- Do not place credentials in frontend build variables, extension files, screenshots, or sample requests.
- Use a restricted key with rotation and monitoring where the provider supports it.
- Redact authorization data from request logs and exception output.

Review and interpretation

Credential handling should be validated with a log review and a configuration audit before release. Rotate the key if it is accidentally committed or exposed.

Threat model focus. The primary risks arise where control or data crosses a boundary: arbitrary browser origins, malformed requests, external prompt content, provider output, credentials, and Gmail DOM selection. This is a prototype-level threat model, not a penetration test. Each risk should map to a specific control and a validation case. In particular, verify that credentials are absent from built frontend files, URLs and logs; that generated text cannot trigger a send action; and that the correct draft receives the result. **Verification detail.** Prioritize mitigations by the type of harm: incorrect context association can expose or misplace content, credential leakage can enable unauthorized provider use, and unclear notices can undermine informed use. Tie each mitigation to evidence rather than a general assurance. **Release consideration.** A threat review should include accidental cross-conversation use, untrusted prompt text, URL-based key exposure, oversized requests, and broad browser permissions. For each risk, identify a concrete check and the evidence that would close it.

D.7 Network transport, CORS, and extension origins

The current clients are configured to call localhost over HTTP. This is convenient for local development, but non-local traffic should use HTTPS. The backend allows every origin through a wildcard CORS annotation; this should be replaced with an explicit policy that reflects the deployed React origin and extension context. The manifest declares the host access needed for Gmail and the local endpoint.

Implementation notes

- Use HTTPS between browser, backend, and provider for deployed environments.
- Keep local development endpoints separate from production configuration.
- Test preflight and extension-origin behavior after narrowing the CORS policy.

Review and interpretation

A deployment checklist should include certificate validity, allowed origins, extension host permissions, and protection against mixed-content failures.

D.8 Prompt injection and generated content

A source email may itself contain instructions that are irrelevant to the requested reply or that attempt to manipulate the model. The current prompt concatenates email content after an instruction without a clear trust-boundary statement. A defensive prompt can instruct the model to treat the email only as context, avoid following embedded commands, and refrain from inventing facts. Prompt wording alone is not a complete security control.

Implementation notes

- Keep the source email clearly delimited from developer instructions.
- Ask for a concise draft and placeholders where information is missing.
- Require user review before external use and provide an easy discard/edit path.

Review and interpretation

Measure refusal, omission, hallucination, and tone behavior using a curated non-sensitive test set. Do not characterize a single example as a safety guarantee.

Input and resource limits. Validation should cover null, blank, very long, and malformed values, not only the normal form submission. A maximum length protects provider cost and service resources, while a tone allow-list prevents arbitrary prompt instructions from being added through the request. Limits should be documented and surfaced to the user before submission. The API should reject invalid input before calling Gemini, and tests should verify both the status returned and the absence of a provider request. **Verification detail.** Boundary tests should include values immediately below and above the accepted limit. Verify that rejected values produce no provider traffic and that users receive a clear correction path without losing the text they entered. **Release consideration.** Acceptance criteria should be phrased as observable outcomes, such as a 4xx response for an unsupported tone or a recoverable error after a provider timeout. Avoid vague criteria like 'secure' or 'reliable' without a measurable condition.

D.9 Logging, privacy notices, and retention

The inspected application does not add a persistence layer for generated replies, but the report cannot infer provider retention settings or hosting logs from the source files alone. A production operator should define what is logged, who can read it, and how long it is retained. The user should receive a clear notice before sending email text to the model service.

Implementation notes

- Prefer metadata such as request status and latency over content-level logs.
- Set retention and access controls for operational logs and rotate logs on schedule.
- Document provider processing, retention, and data residency relevant to the deployment.

Review and interpretation

A privacy review should be completed against the actual deployment, not solely the source repository, and should be revisited when the provider or hosting arrangement changes.

D.10 Request and response contract review

The clients and backend currently exchange a compact JSON request containing emailContent and tone, while the backend returns a raw string. That minimal contract is easy to begin with, yet its null handling, length limits, tone vocabulary, and error representation are not explicit. A documented contract can keep the web interface and extension aligned as the prototype evolves.

Implementation notes

- Define required and optional fields, accepted tone values, encoding, and maximum body size.

- Distinguish successful reply text from validation, provider, and parsing failures.
- Version any incompatible contract change and update both clients together.

Review and interpretation

Treat this as a proposed contract refinement. The current source remains the authority for implemented behavior until the changes are made and verified.

Transport and origin policy. Localhost HTTP is appropriate only for a controlled development setup. A remote deployment should encrypt client-to-service traffic and configure CORS for known web origins; extension access should be limited to the required host permissions. CORS is not authentication and should not be presented as a complete access-control system. Review the browser network request, backend origin checks, and generated extension package as separate evidence items. **Verification detail.** For extension deployments, permission prompts should match the declared host set and the documented feature. Review the packaged manifest rather than only the source copy, and test from an origin that should be rejected. **Release consideration.** The local endpoint should not be hard-coded into a public extension build. Before deployment, verify the configured API host in the compiled web asset and extension package, then check certificate validation and origin behavior in the target browser. **Operational note.** Keep a deletion and correction path for configuration documents if an endpoint or provider credential changes. Historical reports and screenshots should not preserve secret values after rotation.

APPENDIX E - EXTENDED VALIDATION PLAN

E.1 Validation strategy and evidence rules

Validation should progress from deterministic service tests to API integration, browser behavior, and user evaluation. The model provider should be mocked for tests that verify request shape and response parsing; a small number of live requests can separately measure current integration behavior using non-sensitive content. This report records proposed checks only. It does not claim any listed test has been executed.

Implementation notes

- Record the build revision, runtime versions, configuration category, browser version, and test date.
- Use fictional or consented test content with no personal identifiers.
- Attach screenshots or logs only after removing keys, addresses, and private messages.

Review and interpretation

A test result should contain the observed outcome and evidence reference, not merely a planned expected result.

E.2 API contract tests

Controller tests should establish the request and response contract independently from the model provider. A mocked service can confirm that valid values are bound and delegated. Once validation is added, tests should cover missing fields, whitespace, unsupported tones, oversized content, malformed JSON, and consistent status codes. These checks prevent the browser UI from being the only gatekeeper.

Implementation notes

- Valid request returns the documented success schema.
- Invalid request returns a client error without invoking the provider.
- Unexpected service failure returns a stable error identifier rather than raw exception text.

Review and interpretation

Capture request/response examples in the API reference and ensure the frontend handles both success and failure models.

Case	Input condition	Expected behavior
API-01	Valid message and tone	Success response with non-empty draft
API-02	Missing emailContent	Validation error
API-03	Unknown tone value	Reject or normalize per contract
API-04	Payload over size limit	Reject before provider call

Evidence categories. A useful validation plan separates source inspection, deterministic tests, mocked integration, live-provider checks, and browser observations. Each category answers a different question. Source inspection can show that an Axios call exists; it cannot prove network success. A mock can prove parsing behavior; it cannot prove provider availability. A screenshot can show a control; it cannot prove that the draft was inserted correctly. Record these boundaries beside every result.

E.3 Prompt construction tests

Prompt tests should verify how the service combines the base instruction, tone selection, delimiters, and message content. The current default professional wording should be corrected before evaluating Casual and Friendly. Tests can assert structural properties without requiring a live model, including that a missing tone does not add an empty instruction and that the email appears in the designated context section.

Implementation notes

- Check every supported tone and the no-tone path.
- Check quotes, Unicode characters, line breaks, and long message snippets.
- Confirm that prompt wording does not ask the model to create a subject line.

Review and interpretation

Prompt tests should be updated whenever instruction wording, model endpoint, or response format changes.

E.4 Provider response parsing tests

The response parser should be tested against valid output, empty candidate arrays, missing content, missing parts, refusal or blocked responses, malformed JSON, and provider error payloads. The current parser selects candidates[0] and parts[0], so negative cases are especially important. A stable error model lets the client distinguish provider failure from an empty reply.

Implementation notes

- Test one valid candidate with one text part.
- Test absent and null candidate structures without uncaught exceptions.
- Test upstream HTTP error bodies without returning raw secrets or stack traces.

Review and interpretation

Add timeouts and a bounded retry policy only after considering duplicate requests and cost. Store sanitized diagnostics for support.

Prompt test design. Prompt tests should use synthetic messages with clear factual anchors, such as a meeting date or a requested confirmation. Compare tone without allowing the generated text to change

those facts. Include quoted content and an instruction-like sentence to check that email content remains data. A fixture test can verify prompt assembly exactly, while live model evaluation should record model, endpoint, date, and sample conditions. Avoid interpreting one output as proof of general tone quality. **Verification detail.** Keep evaluation prompts stable and record the output criteria before comparing tones. Review factual preservation and edit effort separately; a fluent response is not necessarily a correct or useful response. **Release consideration.** Review outputs with a factual checklist before assigning a tone score. A reply that is friendly but changes a date or promises an action is not acceptable. Record both the useful portions and the edits required, using anonymous case identifiers. **Acceptance detail.** The setup record should state whether the run uses the React client or the extension, because their endpoint configuration and tone behavior are not identical. Include only the command and expected signal needed to reproduce the step.

E.5 React input and form behavior

The UI validation plan covers keyboard use, multiline text entry, tone selection, disabled state, and request lifecycle. At present, the Generate Reply control is disabled for an empty trimmed input, and a blank submit handler also sets a validation message. Test the actual behavior at narrow and wide viewports, including focus order and assistive technology labels.

Implementation notes

- Check empty, whitespace-only, normal, and very long input.
- Check that the tone control selection is included in the request.
- Check button disablement during loading and recovery after success or failure.

Review and interpretation

A visual screenshot is evidence of appearance only; it does not prove that form controls or accessibility behavior work correctly.

E.6 React output, copy, and failure behavior

The result panel should handle an empty server reply, multiline text, long output, and non-string response data. Clipboard permission may be unavailable, so the UI should explain success or offer a selectable fallback. Failure messaging should be actionable without leaking private email text or service secrets. The current implementation shows a generic retry message and does not report copy status.

Implementation notes

- Check newline preservation and long response wrapping.
- Check clipboard success and failure paths.
- Check that a new request clears stale errors and stale output when appropriate.

Review and interpretation

Accessibility checks should verify that loading and error state changes are announced and that focus remains predictable.

Client validation. Web-interface checks should cover the entire state sequence: blank input, valid submission, loading, success, and failure. Confirm that text entered by the user is retained after a network error and that an old reply is clearly separated from a new attempt. Keyboard navigation and copy behavior should be observed in a real browser. If response data changes from raw text to a JSON object, verify that the frontend and extension are updated together. **Verification detail.** Check visible focus and screen-reader status during loading and error recovery, and verify that the result can be selected or copied without unexpected focus movement. These are observable outcomes that can be added to the client case log. **Release consideration.** The test should exercise error recovery as carefully

as the success path: submit valid content, simulate a failed response, then submit again. Confirm that the second result is current and that the progress state and error message reflect the latest request only. **Operational note.** Keep the user-visible error concise and specific enough to distinguish a temporary failure from an invalid request. The case log should state what recovery action worked without retaining the submitted message.

E.7 Gmail extension injection tests

The extension should be tested against a controlled mailbox with one message, multiple messages, new compose windows, replies, forwarded messages, and absent toolbar elements. The MutationObserver should add only one action per relevant compose surface. Selector fallback loops should be corrected so later candidates are evaluated rather than returning on the first miss.

Implementation notes

- Open and close compose windows repeatedly and count injected controls.
- Test multiple simultaneous compose windows without cross-linking message text.
- Test navigation between conversations and verify the captured content belongs to the active one.

Review and interpretation

Record Chrome version and Gmail layout date because DOM changes may alter integration behavior without source changes.

E.8 Gmail content extraction and insertion tests

Extraction tests should confirm exactly which visible message text is sent, including quoted history and signatures. Insertion tests should confirm the response appears at the intended cursor position in the active draft and does not overwrite existing text. The current code uses document.execCommand('insertText'), which may behave differently as browser support evolves.

Implementation notes

- Confirm the extracted text is not empty when a supported message element exists.
- Confirm insertion does not replace the user's existing draft or send the message.
- Verify failure to find the compose box produces a visible recovery option.

Review and interpretation

Prefer supported editor or mail APIs if available, and retain a copy/paste fallback for DOM integration failures.

Gmail validation. Gmail testing must account for multiple compose surfaces and dynamic page mutations. Use a test mailbox with a new message, an existing reply, and two simultaneous compose windows. Verify that the action appears once in the correct toolbar, that extraction is limited to the intended conversation, and that insertion preserves pre-existing draft text. Keep screenshots free of names, addresses, and private message content. **Verification detail.** Test the extension with a draft that already contains text and with multiple open compose windows. The expected outcome is that only the intended editor changes, with a clear indication that the inserted text is still a draft. **Release consideration.** Cover selector fallbacks with a fixture or controlled browser page where the primary selector is absent. Record the active conversation and editor chosen. If the expected element cannot be located, the safest outcome is a visible failure with no draft change. **Acceptance detail.** Document whether each browser result was observed on a new message, reply, or forwarded conversation. These page structures can differ, and the content script should not combine text from an unrelated open compose window.

E.9 Network and resilience tests

The clients should be checked under backend unavailability, slow responses, abrupt disconnects, invalid JSON, and model quota errors. The React spinner should eventually stop, the user should see a safe error, and the backend should not leave worker threads blocked indefinitely. The extension should restore button text and disabled state in all outcomes, which its finally block currently attempts to do.

Implementation notes

- Test connection refused and timeout conditions.
- Test server 4xx and 5xx responses.
- Test repeated clicks and duplicate in-flight requests.

Review and interpretation

Use explicit timeout budgets and monitor latency distributions before tuning retries. Avoid automatic retries that can multiply provider usage unexpectedly.

E.10 Security and privacy validation

Security tests should verify that no credential is present in frontend assets, browser network URLs, source control, logs, or screenshots. Confirm that only authorized origins can call the API, requests are encrypted in deployment, payload limits are enforced, and the privacy notice accurately describes provider processing. Test prompt injection with synthetic content only.

Implementation notes

- Inspect built JavaScript and extension package for secrets.
- Review redacted server logs after success and failure cases.
- Check access-control and rate-limit behavior from an untrusted origin.

Review and interpretation

Use a qualified security review for production deployment; this report is not a penetration test or privacy certification.

Resilience evidence. Network and resilience checks should distinguish connection refusal, slow provider response, malformed JSON, provider quota failure, and empty model output. Each case should end with a bounded user wait and a stable failure outcome. Do not add retries until duplicate-call behavior and cost are understood. Record the elapsed time, status category, client state after failure, and whether sensitive provider details were withheld. **Verification detail.** Use a bounded wait in each client and confirm the progress indicator always stops. Record if an error is recoverable and whether a retry is an explicit action; do not treat a quiet timeout or indefinite spinner as a successful completion. **Release consideration.** A bounded request also needs a bounded UI wait. Record the configured timeout, observed recovery time, and client state after a timeout. If the service later adds retries, test the maximum number of provider calls and make the retry visible to the user. **Acceptance detail.** Keep a clear distinction between an observed local behavior and a recommended control. For example, the current report can identify the local HTTP endpoint from source, while HTTPS readiness can only be claimed after the deployment is configured and verified.

E.11 Usability and quality evaluation

A user study could compare the time and effort to draft a reply manually with the time to review and revise an AI proposal. Participants should use representative, non-sensitive tasks and rate relevance, tone, correctness, edit effort, and trust. The report should state the participant count, recruitment, task order, and limitations rather than presenting anecdotes as generalizable results.

Implementation notes

- Track completion time and number of edits without collecting actual private mail.
- Ask whether the user understood that the text was model-generated.
- Record unsupported facts or tone mismatches as distinct error categories.

Review and interpretation

Any conclusion about productivity improvement requires measured data and an appropriate baseline; no such measurement was supplied for this report.

E.12 Results log template and acceptance gate

Use the following result sheet when the proposed cases are run. Acceptance should require evidence for the happy path, validation, upstream failure, extension recovery, and privacy controls. A case marked blocked or not run should remain visible in the final submission rather than being silently counted as passed.

Implementation notes

- Record case ID, environment, actual result, pass/fail/blocked state, evidence file, and defect ID.
- Re-run affected cases after a fix and preserve the earlier result for traceability.
- Review unresolved high-impact defects before any public or institutional demonstration.

Review and interpretation

This report's matrix remains a plan. Fill in the actual observations only after running the relevant application and retain the evidence securely.

Case ID	Observed result	Status	Evidence / defect
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

Evaluation limits. Usability and quality conclusions require measured observations. A controlled study should define tasks, participant criteria, baseline drafting conditions, success measures, and privacy protections before collecting data. Possible measures include review time, edit count, factual error rate, tone fit, and user understanding of model limitations. Without such data, this report can describe an evaluation method but cannot claim productivity gains or user acceptance. **Verification detail.** If a study is later run, report aggregate findings and limitations without publishing participant messages. Separate perceived usefulness from correctness and include cases where users rejected or substantially edited the generated text. **Acceptance detail.** A privacy review should confirm what appears in application logs, reverse-proxy logs, provider dashboards, and evidence folders. Retention may differ across these systems, so one setting cannot stand in for a complete data-handling description.

APPENDIX F - MAINTENANCE AND HANDOVER

F.1 Change management and version traceability

The repository's Git history can provide the authoritative code revision for a release, but no milestone or change log was included in the supplied report materials. A maintainable project should associate each report or demonstration build with a commit identifier, dependency lockfiles, and a brief release note. Changes to model prompts and APIs deserve explicit tracking because they can alter generated behavior without changing the visible interface.

Implementation notes

- Record the source revision and build date for every demonstration.
- Review package updates and backend dependency updates deliberately.
- Note prompt and provider-model changes in release documentation.

Review and interpretation

Keep reports and screenshots linked to the source revision they describe. Do not reuse an old screenshot to represent materially changed behavior.

F.2 Build and deployment packaging

The web interface can be built by Vite, the backend by Maven, and the Chrome extension from its manifest and content files. The current README references folders and a React extension structure that do not exactly match the source tree. Packaging instructions should be corrected, and development URLs should be replaced with deployment configuration. Secrets and user data must remain outside public build assets.

Implementation notes

- Build each component from its actual directory.
- Pin or review dependency versions and archive the generated build outputs.
- Verify the extension manifest's host permissions before packaging.

Review and interpretation

A release package should include configuration examples with placeholders only, version identifiers, and a concise installation and rollback procedure.

Change traceability. A report should describe the same source revision that was reviewed for screenshots, diagrams, and implementation claims. Prompt edits and dependency changes can alter behavior without changing the UI, so capture the revision and build context with each evidence artifact. The repository does not include formal release records; introduce a simple change log before handover rather than reconstructing dates or owners from memory. **Verification detail.** When a prompt or dependency changes, rerun the cases that cover the changed behavior and identify the previous baseline. Keep screenshots and generated artifacts linked to the new revision so they do not imply behavior from an older build. **Release consideration.** Change records should identify why a release was made and which acceptance cases were rerun. Include prompt text, dependency updates, endpoint changes, and extension selector changes because each can alter behavior independently of the interface. **Acceptance detail.** A prompt case should include a source message with a concrete fact and a quoted instruction-like phrase. Review whether the model follows the requested tone while preserving the fact and treating quoted text as context, not application guidance.

F.3 Operational monitoring and support

A production service needs availability, latency, and error monitoring to distinguish client failures from provider failures. Metrics should avoid message content and credentials. Health endpoints, request correlation identifiers, and sanitized error categories can improve diagnosis. The current source includes Actuator as a declared dependency in the README but the checked pom.xml dependencies should be reconciled before relying on an Actuator endpoint.

Implementation notes

- Track request count, latency, upstream status category, and error rate.
- Set alert thresholds and define an owner for provider credential rotation.
- Keep logs access-controlled and retain them only as long as needed.

Review and interpretation

Monitoring should not turn a privacy-sensitive drafting service into a repository of the user's email content.

F.4 Accessibility and inclusive interaction

The React interface uses Material UI form controls with visible labels and status elements, but accessibility must be verified in an actual browser. Check keyboard navigation, visible focus, screen-reader names, error announcement, contrast, and zoom. Gmail injection should use a control with an accessible name and a clear progress state. The current DOM button is built as a div with role=button, so keyboard support needs review.

Implementation notes

- Confirm all actions work by keyboard alone.
- Announce loading, success, and errors to assistive technology.
- Provide an accessible name and keyboard activation for the Gmail action.

Review and interpretation

Accessibility findings should be recorded as implementation issues and rechecked after each UI change.

Package integrity. Build each client from its actual directory and inspect the output before release. The frontend bundle should contain an environment-specific service URL but no provider key. The extension package should include only the intended manifest permissions and source files. Preserve the backend configuration template separately from real secret values. A clean checkout build is stronger handover evidence than a successful build using an undocumented local environment. **Verification detail.** Inspect the final package for local URLs, debug output, stale permissions, and accidental secrets. Archive the versioned output and configuration template together so another developer can reproduce the same release without receiving private credentials. **Release consideration.** A release archive should include the extension manifest, built web assets, backend package, configuration template, and revision identifier. Keep secret values in a separate protected store and ensure they are not accidentally bundled with the archive. **Acceptance detail.** Check both the normal Gmail toolbar and a layout where the expected toolbar is delayed or absent. The extension should retry a bounded discovery process or stop with a safe status; it should not create controls in unrelated page regions.

F.5 Roadmap and release phases

A reasonable roadmap begins with correctness and reliability, then privacy and deployment, followed by usability research and optional product features. Fix the prompt and response parser before adding additional tones. Add server-side validation, controlled CORS, HTTPS, timeouts, and error mapping before a broader deployment. Only after these steps should the project assess language selection, user preferences, or mailbox-provider expansion.

Implementation notes

- Phase 1: stabilize request validation and response behavior.
- Phase 2: harden credentials, transport, access controls, and privacy messaging.
- Phase 3: test and improve Gmail integration; measure usability with consented users.

Review and interpretation

Each phase should have acceptance criteria, an owner, and evidence. Avoid describing roadmap items as present capabilities.

F.6 Submission and handover checklist

Before institutional submission, complete academic details on the bonafide certificate and attach the approved similarity report. Confirm that the report's author and project title are correct and that the two screenshots match the described workflow states. Execute the planned validation cases and update Chapter 4 with the real outcomes rather than the current 'not executed' status.

Implementation notes

- Fill supervisor, enrollment, examiner, and submission date details using verified information.
- Keep the supplied app and Gmail screenshots tied to the described workflow states.
- Attach the similarity report and reconcile references, page numbers, and release revision.

Review and interpretation

Two supplied screenshots are included: the web application initial state and the Gmail compose view with the AI Reply control. Neither image demonstrates a live Gemini generation or verified draft insertion.

Support and accessibility. Operational support should be designed around sanitized error categories and minimal telemetry. Request identifiers can help correlate a client failure with a backend event without storing the email text. Accessibility checks should accompany functional checks: keyboard activation, visible focus, accessible labels, status announcements, zoom, and contrast. The extension's current div-based control needs particular scrutiny because pointer visibility alone does not establish keyboard operability. **Verification detail.** A support path should ask for a case identifier, client path, and sanitized error category rather than a copied email. Accessibility findings should be recorded as defects with repeatable steps and verified again after changes. **Release consideration.** Support staff should have a redaction-safe diagnostic path and a contact for accessibility issues. Record defects with reproducible steps, affected client, and expected behavior so they can be fixed without collecting the user's source email.

APPENDIX G - PROJECT REQUIREMENTS AND DESIGN RECORD

G.1 Scope and actors

SmartMail Assistant has two client entry points: a standalone React page and a Chrome content script intended for Gmail. The person composing the message is the primary actor; the backend and Gemini endpoint are supporting services. The report treats the system as a draft assistant, not an autonomous mail agent.

Implementation notes

- The user supplies source text and may select a tone in the web client.
- The backend prepares the request and returns generated text.
- The user remains responsible for reviewing and deciding how to use the draft.

Review and interpretation

Any feature outside these boundaries should be recorded as future work until it is present and verified.

G.2 Functional requirement map

Requirements can be traced from visible controls through the API to the provider. This map helps reviewers separate a feature request from evidence that the supplied source implements it.

Implementation notes

- FR-01: accept non-empty email text in the React client.
- FR-02: submit email text and tone to the configured API.

- FR-03: display a returned reply and provide a copy action.
- FR-04: expose an AI Reply action in Gmail and attempt draft insertion.

Review and interpretation

Trace each requirement to a source file and a test case; mark evidence as source-inspected or executed.

Requirements and evidence. Requirements should be traceable to code and to a verification method. For example, 'display a reply' maps to the React response panel and a client test, while 'insert into Gmail' maps to selector resolution, editor targeting, and a browser test. Mark each item as source-inspected, executed, blocked, or future work. This avoids turning a project objective into an unsupported claim of completed behavior. **Verification detail.** A requirement should have one or more acceptance conditions that can be observed. For example, a 'reviewable reply' should appear as editable text and remain unsent; the requirement is not satisfied by a successful provider response alone. **Release consideration.** A trace entry should cite the source path or test identifier and describe what that evidence actually proves. For example, a component source can establish a copy handler exists; a paste observation establishes that the clipboard action works in a browser. **Acceptance detail.** Use observable acceptance criteria for the functional requirements: a request is sent with the chosen tone, a response is displayed or inserted in the intended client, and the user can review it before use. Keep each criterion mapped to its own evidence. **Evidence note.** The final record should indicate whether a requirement is complete, partially represented in source, or awaiting a test. This simple status keeps the chapter's design intent tied to the validation plan.

G.3 Quality attribute priorities

The prototype's most important quality attributes are correctness of the draft workflow, privacy of message content, resilience to provider failure, and usability across both clients. These goals can conflict: richer logging may help diagnosis but expose sensitive text.

Implementation notes

- Correctness: preserve user input and do not silently send generated text.
- Privacy: minimize transmitted content and avoid message-body logs.
- Reliability: bound request time and recover the UI after errors.
- Usability: explain loading, errors, and review responsibility.

Review and interpretation

Prioritize controls using the deployment context and the sensitivity of the messages users may submit.

G.4 Input and output data dictionary

The present request contains emailContent and tone. The web UI returns generatedReply to component state, while the backend response is a raw text value. A dictionary clarifies the role and handling expectations for each field.

Implementation notes

- emailContent: user-provided source message text; sensitive by context.
- tone: optional text value; client options include Professional, Casual, and Friendly.
- generated reply: model-produced draft; treat as unverified text.
- error state: user-facing status; avoid embedding raw secrets or provider URLs.

Review and interpretation

The current DTO does not express all proposed validation constraints; this dictionary is a design aid.

Quality priorities. Quality attributes compete in this prototype. More diagnostic detail can improve support but increase exposure of email text; more aggressive retries can improve apparent availability but repeat provider cost; and broader permissions can make extension matching easier but expand access. Make these trade-offs explicit and choose controls according to the deployment and message sensitivity. Reassess the priorities when the system moves beyond a local demonstration. **Verification detail.** Document the reason for each quality priority and revisit it when the audience or hosting environment changes. A local demonstration with synthetic data has different exposure than a shared service used for real correspondence. **Release consideration.** Choose priorities in relation to the deployment: a local classroom demonstration can tolerate different availability controls than a shared service handling confidential mail. In either case, do not relax credential protection or human review of generated text. **Acceptance detail.** Availability can be measured by successful synthetic requests, but that measure says nothing by itself about reply accuracy. Privacy can be evaluated through data-flow and logging review, while usability needs direct user evidence. Keep these quality measures separate in the design record. **Review note.** For the prototype, prioritize privacy and correctness before optional personalization. A usability improvement should not make the provider path less visible or weaken the user's ability to inspect and edit the draft.

G.5 Use case: generate from the web form

The normal web use case begins with the user opening the page, entering a message, selecting a tone, and requesting a reply. The output is shown for review and may be copied to another application.

Implementation notes

- Precondition: the web client and backend are available and provider settings are valid.
- Main path: validate non-blank input, post the request, show progress, render returned text.
- Alternate path: display a generic failure message when the request fails.

Review and interpretation

Acceptance should verify both successful response handling and recovery from failures.

G.6 Use case: generate from Gmail

The extension use case is intended to reduce context switching by placing an AI Reply control in a Gmail compose surface. It reads visible message text, calls the local API, and attempts to insert the result into the draft.

Implementation notes

- Precondition: extension is loaded and Gmail page selectors match.
- Main path: locate compose controls, invoke generation, and place text in the editor.
- Safety condition: the action must not trigger Gmail Send.

Review and interpretation

The supplied image documents the visible action control, not a verified backend result or insertion outcome.

Use-case acceptance. The two user paths share a generation service but differ at the point of input and output. The web use case begins with user-entered text and ends with a visible, copyable reply. The Gmail use case begins with page extraction and ends with attempted insertion into a compose editor. Their acceptance conditions should therefore be separate: correct field handling for the web form, and correct conversation/editor scoping for Gmail. **Verification detail.** For the web flow, acceptance ends at a correct, visible draft and a usable copy action. For Gmail, acceptance includes context extraction

and editor selection. Keep the test scripts distinct so success in one client is not generalized to the other. **Release consideration.** For the web path, capture the displayed result and clipboard behavior; for Gmail, capture the control and separately verify editor text. Keep the acceptance records separate so a client that was not exercised cannot inherit the other client's pass status. **Acceptance detail.** The web scenario should include a recovery path: if generation fails, the entered message remains and the user can correct the tone or retry. The Gmail scenario should instead protect the existing draft and expose a safe alternative such as manual copy. **Review note.** An alternate web path should cover provider failure after the user has entered a valid message. The expected result is a clear status, retained input, and no stale draft presented as current. **Evidence note.** Specify what the user can do after failure and what state must remain unchanged. A recovery path is part of the use case, not an optional implementation detail.

G.7 Requirement traceability worksheet

The following worksheet structure can connect report claims to evidence. Keeping this trace helps future revisions avoid calling a planned behavior implemented.

Implementation notes

- Requirement ID and description.
- Implementation location and observed source behavior.
- Verification case, evidence artifact, result, and revision.

Review and interpretation

A requirement without a test or source reference should remain an open item.

G.8 Non-functional constraints

Several constraints follow from the architecture even though they are not fully implemented. Provider availability, network latency, browser DOM changes, and credential configuration all affect the user-visible result.

Implementation notes

- Generation depends on network access to the backend and Gemini service.
- The extension depends on Gmail DOM structures that can change.
- The local endpoint and fixed configuration are suitable only for development demonstration.

Review and interpretation

Document each environment boundary before a public demonstration.

Traceability and constraints. A traceability worksheet should capture the exact implementation location and the evidence that supports each statement. Constraints such as network availability, provider quotas, local endpoint configuration, and Gmail markup are not edge cases; they define the operating envelope. Record them with the requirement so reviewers understand where the prototype is expected to work and where additional engineering is necessary. **Verification detail.** The trace should identify whether evidence comes from source, a unit test, a mocked integration, or a live browser observation. This makes the report reviewable and prevents a proposed control from appearing as an implemented fact. **Release consideration.** The constraint register should name which client is affected, what breaks when the condition is unmet, and how a user recovers. This turns limitations into operational information rather than a general list of technology caveats. **Acceptance detail.** The operating envelope should include supported runtime versions, provider availability, browser host access, and local or deployed API location. When a condition changes, identify the affected requirement and retest the client path that depends on it. **Review note.** The same limit should be enforced by the service even when a client has no visible character counter. Document the accepted range only after it

is chosen and implemented; do not invent a current limit from the report template. **Evidence note.** Keep scope notes versioned so future contributors can see when account storage, history, or other out-of-scope behavior is introduced and what new controls it requires. **Operational note.** A scope worksheet should include the reason a capability is excluded and the evidence that would be required before it is reconsidered. This keeps the boundary useful during feature planning.

G.9 Out-of-scope capabilities

The source review did not establish account management, message history storage, automatic email delivery, analytics, or attachment processing. Stating this boundary prevents readers from inferring capabilities from the product concept.

Implementation notes

- No automatic send action is described in the reviewed clients.
- No application database or user-account workflow is identified in the supplied files.
- The extension interacts with page markup rather than a Gmail API integration.

Review and interpretation

Revisit scope only when corresponding implementation and verification evidence exist.

G.10 Assumptions and dependencies

A local demonstration assumes that the correct runtimes are installed, the provider URL and key are configured, and the backend is reachable from the client. Gmail integration adds browser permission and page-structure assumptions.

Implementation notes

- React dependencies install successfully in the frontend directory.
- Java and Maven can build and launch the backend.
- Provider account, quota, and endpoint are valid for the configured model.

Review and interpretation

Keep assumptions visible in release notes and setup instructions.

Assumptions and scope control. Assumptions should be tested at setup rather than left implicit: runtime versions, provider credentials, network reachability, browser permissions, and the current Gmail page structure. Out-of-scope items such as accounts, stored history, analytics, automatic sending, and attachment processing should remain explicit. This prevents readers from inferring a broader product from the assistant concept or from the presence of a browser extension. **Verification detail.** A scope change should trigger review of the data inventory, permissions, interface notice, and tests. Adding history or accounts, for example, would introduce storage and access-control requirements that are not present in the current project. **Release consideration.** Keep assumptions in setup and release documents where they can be checked. If provider credentials, browser permissions, or Gmail structure differ from the expected environment, stop the demonstration and document the difference instead of treating it as a user error. **Acceptance detail.** If one component changes its contract, identify downstream callers and update them in the same release. The API service is shared, but the React page and extension handle the response differently; each needs a client-specific check. **Review note.** A responsibility matrix can be used during code review: frontend changes require web-state checks, service changes require contract and parser checks, and extension changes require a real browser test. Shared changes require all affected checks. **Evidence note.** For an assumption that cannot be checked at setup, record a risk and an owner instead of leaving it as an unverified prerequisite.

G.11 Risk register

A compact risk register makes important technical and privacy risks actionable. Ratings should be revisited after implementation evidence is collected rather than treated as fixed facts.

Implementation notes

- R1: provider or network failure prevents generation; mitigate with bounded timeouts and recovery.
- R2: key exposure through URL or logs; mitigate with secret handling and redaction.
- R3: Gmail selector change breaks insertion; mitigate with browser regression checks.
- R4: unsupported model claims; mitigate with user review and prompt constraints.

Review and interpretation

Record owner, likelihood, impact, mitigation, and residual risk for each release.

G.12 Design decision record

A decision record captures why the prototype uses a backend between browser clients and Gemini. This separation keeps provider credentials out of ordinary client code, but it does not by itself guarantee safe deployment or prevent exposure through request logs.

Implementation notes

- Decision: both client paths call a shared server endpoint.
- Benefit: one place to construct prompts and manage provider communication.
- Trade-off: backend availability and deployment configuration become required.

Review and interpretation

Re-evaluate decisions when adding authentication, additional clients, or production hosting.

Risk ownership. A risk register becomes useful when each item has an owner, a trigger, a mitigation, and residual status. For example, a provider failure can be detected by timeout cases; credential exposure can be checked in builds and logs; and selector drift can be checked in browser regression cases. Ratings should be assigned by the project team using its context. This report identifies risk categories but does not invent likelihood scores or completed mitigations. **Verification detail.** Close a risk only when the mitigation is implemented and its evidence is recorded. A planned timeout or proposed secret store is not a completed mitigation; retain the residual risk until the relevant configuration and test exist. **Release consideration.** Review risk ownership at each handover, especially credential rotation, provider quotas, and browser compatibility. A mitigation that depends on an unnamed owner is unlikely to remain effective after the original developer leaves. **Acceptance detail.** For the extension state model, specify what the user sees while the request is pending and how the action returns to ready after an error. Prevent a duplicate click from creating a second request, but do not leave the control disabled indefinitely. **Review note.** State invariants include one active request per action, a visible indication of pending work, and no automatic send. Write these invariants beside the model so that new states do not accidentally undermine draft safety. **Evidence note.** A residual risk should include the limitation that remains after a mitigation is applied. This prevents a partial fix from being mistaken for complete protection.

APPENDIX H - INTERACTION AND COMPONENT DESIGN

H.1 Component responsibility matrix

The interface, controller, service, and extension each perform distinct work. Clear ownership reduces duplicated validation and ambiguous error handling.

Implementation notes

- React App: collect fields, manage loading/error state, show and copy output.
- Controller: bind request and delegate generation.
- Gemini service: construct prompt, call provider, parse candidate text.
- Content script: find Gmail page elements, request reply, attempt insertion.

Review and interpretation

A future modular design can retain these boundaries while adding typed contracts.

H.2 Web request sequence

The web sequence starts at the form and ends when the returned reply becomes visible. Error and loading transitions are part of the sequence and should be examined alongside the happy path.

Implementation notes

- User activates Generate Reply after entering content.
- React posts a JSON request to the backend route.
- Service sends a prompt to Gemini and parses provider output.
- React renders the result or a user-facing error.

Review and interpretation

Timing evidence should distinguish client, backend, and provider delays where instrumentation permits.

Component contract. Component boundaries should make error ownership clear. The controller is responsible for request binding and HTTP outcomes; the service owns prompt construction, provider transport, and response parsing; clients own interaction feedback and draft review. Moving validation into the API does not remove the need for client checks, but it ensures direct callers receive consistent behavior. A stable contract enables the two clients to evolve without duplicating provider logic. **Verification detail.** Keep a single API contract source or versioned specification as the clients evolve. Without it, UI assumptions about response text and server assumptions about tone can drift independently and create failures that appear only in one client. **Release consideration.** The API contract should be the shared reference for both client implementations. A small change log with request and response examples can prevent the React client from accepting a different shape than the extension expects. **Acceptance detail.** A prompt specification should define the role, tone precedence, source-message boundary, and output shape. Use one base instruction and a validated tone value; do not append arbitrary user-supplied instructions as if they were a supported tone. **Review note.** Prompt examples should use redacted synthetic text and show how the tone value is applied without placing it inside the source-message block. This makes prompt review safer and easier to compare across revisions.

H.3 Gmail extension sequence

The extension sequence begins with a page mutation that makes a compose toolbar available. A content script adds an action, reads message content, requests the service, and updates the compose editor.

Implementation notes

- Observe page changes and identify the intended compose container.
- Create an accessible AI Reply action and prevent duplicate controls.
- Capture only the active conversation context required for drafting.

- Insert the result without sending or overwriting unrelated draft text.

Review and interpretation

DOM interaction must be tested after Gmail UI changes and with multiple compose windows.

H.4 State model for the web form

The web form can be reasoned about as idle, invalid, submitting, success, or error states. The React implementation has input, tone, generated reply, loading, and error state values; not all transitions are presented as a formal state machine.

Implementation notes

- Idle: no request is active.
- Invalid: input is blank or whitespace-only.
- Submitting: disable duplicate submission and show progress.
- Success or error: show result or recovery guidance and restore controls.

Review and interpretation

State transition tests help identify stale errors or old output left visible after a new request.

Sequence and timing. The request sequence includes delays at several layers: user interaction, browser network, API processing, provider response, and client rendering. A single end-to-end stopwatch cannot identify which layer needs improvement. Instrument only non-sensitive timing and status data, and use synthetic messages. Include a failure branch in diagrams so that timeout and parse errors are visible design outcomes rather than exceptional details omitted from the model. **Verification detail.** A sequence diagram should label which component owns each timeout and error translation. This helps reviewers see where loading may persist and where content crosses from the browser to the server and provider. **Release consideration.** Instrument only the data needed to answer operational questions. Request duration, safe status category, and correlation ID are often sufficient; message content and prompt text are not necessary for routine latency analysis. **Acceptance detail.** An environment record should use categories for provider credentials and URLs rather than copying their values. Store test artifacts under access control and keep a clean version for institutional submission with no account identifiers. **Review note.** Use a run identifier and case ID to connect a result to its evidence while keeping the actual email body out of the log. A reviewer should be able to reproduce the test from the fixture and environment description.

H.5 State model for the extension action

The Gmail action should move from ready to generating and then to inserted or failed. In the current code, the button label and disabled state are managed around a request, but the actual interaction depends on selectors and editor behavior.

Implementation notes

- Ready: the correct message and compose surface are identified.
- Generating: prevent duplicate clicks and signal progress.
- Inserted: make the inserted text clear to the user.
- Failed: restore the action and explain a safe retry path.

Review and interpretation

Do not equate button presence with successful message extraction or insertion.

H.6 Sequence diagram notation guide

A sequence diagram should show the user, selected client, backend controller, generation service, and Gemini endpoint as separate lifelines. The response should travel back through the same layers before it is displayed or inserted.

Implementation notes

- Label the JSON request fields and the provider prompt separately.
- Show an alternate branch for provider error and parse failure.
- Mark the point at which email content leaves the browser trust boundary.

Review and interpretation

This diagram is a documentation artifact; it is not execution trace evidence.

State transition review. State diagrams are useful when they describe observable behavior rather than merely restating variable names. Each transition needs a trigger and an outcome: blank submission stays local; valid input enters loading; success displays a draft; failure communicates recovery; a new attempt resets stale status. The extension should add equivalent generating, inserted, and failed states, even if its current implementation represents them through button properties. **Verification detail.** Test transitions with repeated activation and quick input changes, not only one click at a time. The purpose is to detect duplicate requests, stale results, and status indicators that describe an earlier request. **Release consideration.** State transitions should include a repeated-click case and a failed request followed by a successful retry. These cases reveal whether multiple calls overlap or whether stale output is presented after the application has moved to a new state. **Acceptance detail.** For malformed provider output, verify the exact client outcome for no candidates, missing parts, empty text, invalid JSON, and a provider error status. Each case should finish without a stack trace in the UI or error content being mistaken for a reply. **Review note.** Test parser recovery with an empty string and with a response that contains text in a later candidate. The current code selects the first candidate and part, so the expected handling must be explicit before parser behavior is changed. **Handover note.** Document the control's pending label and the safe outcome when the request finishes after its compose window closes.

H.7 Component interaction and trust boundaries

The application has a browser-side trust area, a backend service area, and an external provider area. Gmail page content and model output are both untrusted inputs for different reasons.

Implementation notes

- Validate user input at the server boundary.
- Keep provider credentials on the backend and redact them from diagnostics.
- Treat generated text as data to review, not as an instruction to execute.

Review and interpretation

Deployment diagrams should add the actual hosting, TLS, firewall, and logging components.

H.8 API endpoint specification

The source route is POST /api/email/generate. The request object has emailContent and tone fields; the response currently is a raw text string. A stable API spec should define examples and errors before external clients depend on it.

Implementation notes

- Document content type, required fields, maximum sizes, and tone vocabulary.

- Set explicit response examples for success and each failure category.
- Define timeout behavior and whether repeated requests are idempotent.

Review and interpretation

Treat examples as a proposed specification until adopted in code.

Draft safety. The draft is a user-controlled artifact. Insertion should be scoped, reversible, and visible, especially when the editor already contains text. A test should compare the editor before and after the action and confirm that unrelated compose windows remain unchanged. The AI control must not overlap or activate Gmail's standard send action. These checks are more important than visual placement alone because they protect message context and user intent. **Verification detail.** Make insertion behavior explicit when a draft contains a selection or cursor position. Preserve the user's existing content and never send; verify the result by comparing the editor's text before and after the action. **Release consideration.** If the insertion target is uncertain, stop and let the user copy the result through a safe path. Guessing at an editor can place private or incorrect content in the wrong draft, which is a higher-impact failure than requiring an extra user action. **Acceptance detail.** The Gmail case matrix should include an existing draft, multiple open windows, and a conversation with quoted history. Verify that only the intended editor changes and that Send remains untouched; preserve a redacted before-and-after record. **Review note.** Use a test message with quoted history to check that the extraction selects intended context and avoids unrelated page text. Record the selected content length or a synthetic hash rather than saving the message itself. **Evidence note.** A component review should confirm that page text cannot cross into another compose surface and that model output is never executed as markup or an application command. **Operational note.** When the model response returns after a compose surface closes, discard it or present it without modifying another draft.

Confirm this behavior under delayed responses before describing the extension as reliable.

H.9 Prompt assembly specification

The current service creates a professional email reply instruction, adds a tone instruction when supplied, and appends the original email. A more explicit contract can reduce conflicting tone directions and constrain unsupported claims.

Implementation notes

- Use one base instruction and one validated tone instruction.
- Mark source email as quoted data, not trusted instructions.
- Ask for concise text and no fabricated facts; preserve unknown details as placeholders.

Review and interpretation

Evaluate prompt revisions using synthetic cases and record model/version conditions.

H.10 Provider response schema

The service selects the first candidate and first text part from the provider JSON. A schema checklist can identify missing branches that should be handled before deployment.

Implementation notes

- Success: at least one candidate with a text part.
- No candidate, empty content, or blocked response: controlled error.
- HTTP error, malformed JSON, or timeout: stable client-visible failure.

Review and interpretation

Provider response formats can change; validate against current documentation before updating the parser.

API and prompt contract. The endpoint contract should document what the backend accepts and how the provider instruction is formed. Keep tone values separate from raw user text, enforce a known vocabulary, and delimit the source message. Provider response parsing should be tested against success and failure fixtures. If the contract becomes a JSON envelope, update React and the extension in the same change and add a compatibility note for any external caller. **Verification detail.** Changing the API contract should include a migration note and coordinated client release. If both clients cannot be updated atomically, use an intentional versioning strategy rather than silently changing the response format. **Release consideration.** Include a contract example for valid JSON, blank input, unsupported tone, provider timeout, and empty response. Keep the example consistent with the actual controller status mapping once that mapping is implemented. **Acceptance detail.** A configuration matrix should distinguish local, staging, and production values without embedding secrets. Confirm that the API URL, allowed origins, provider key source, and extension host permissions are correct for the target environment before packaging. **Review note.** Before packaging, verify the configuration loaded by the running backend and the endpoint embedded in each client. Keep screenshots of configuration screens out of the submission if they reveal keys or account identifiers. **Operational note.** Use actual deployment values only in protected configuration. The report and example matrix should show placeholders and the names of settings, never a live URL containing a key.

H.11 UI layout and accessibility map

The React interface presents a title, multiline input, tone selector, submit action, progress indicator, generated text, and copy action. The visual order should match keyboard and screen-reader reading order.

Implementation notes

- Give every form control a programmatic label.
- Announce progress and errors without moving focus unexpectedly.
- Support zoom, narrow viewports, keyboard use, and visible focus.

Review and interpretation

The supplied initial-state screenshot documents appearance only and is not an accessibility test.

H.12 Extension selector inventory

The content script relies on selectors for the compose toolbar, message text, and editable compose box. A selector inventory records why each candidate is used and how failure is handled.

Implementation notes

- List current primary and fallback selectors with source evidence.
- Test every fallback path; review loops that may return after the first miss.
- Avoid silently choosing an unrelated message or compose surface.

Review and interpretation

Do not depend on private or undocumented DOM structure without monitoring its fragility.

Selector and accessibility review. Selector inventories need a primary selector, tested fallback, scoping rule, and failure behavior. A broad page query can select the wrong conversation, while early returns can prevent fallbacks from running. The injected control should also be semantically interactive: use button behavior, accessible name, focus treatment, and keyboard activation. Recheck both semantics and selectors after Gmail or browser updates. **Verification detail.** A selector failure should stop safely

and leave the user's draft unchanged. Keep fallbacks narrow, test each one, and avoid relying on element ordering when several conversations or compose windows are visible. **Release consideration.** Document the selector's container scope and the expected accessible role for each control. During regression, verify that repeated mutations do not create duplicate actions and that keyboard activation triggers the same safe request path. **Acceptance detail.** Ownership should cover credential rotation, provider quota review, endpoint availability, extension compatibility, and privacy inquiries. Assign roles from the actual organization and document a contact path that does not request a user's full email content. **Review note.** Operational ownership should include a backup contact and a procedure to disable the affected client path. For a provider outage, users should retain a manual drafting option and should not be asked to resend private content to support. **Evidence note.** Accessibility and selector checks should be repeated in the target browser after a Gmail update, because DOM changes can affect both control discovery and keyboard behavior. **Operational note.** A reviewer should be able to see how a browser action maps to its API request and how the result returns to the same draft. Record both the control semantics and the data scope.

APPENDIX I - DETAILED VALIDATION AND EVIDENCE WORKSHEETS

I.1 Test environment record

Every executed validation case should identify the environment sufficiently to reproduce the result without recording sensitive email content. Redact credentials and personal identifiers from evidence.

Implementation notes

- Record source revision, operating system, Java, Node, browser, and dependency versions.
- Record client path, backend URL category, provider/model configuration category, and network condition.
- Use a synthetic message ID rather than storing the actual message body.

Review and interpretation

A blank environment field should be treated as a reproducibility limitation.

I.2 Test case: blank input

This case confirms that empty and whitespace-only input is stopped in the web client and rejected by the API if submitted directly. The repository currently shows a web-side check but no equivalent server validation annotations.

Implementation notes

- Prepare empty and whitespace-only values.
- Attempt the normal UI flow and a direct API request.
- Record client response, HTTP status, and whether a provider call occurred.

Review and interpretation

Expected server-side rejection remains a proposed requirement until implemented.

Reproducible test conditions. A test result is reproducible only when its environment is described. Record the commit, browser, Java and Node versions, configuration category, and whether Gemini was mocked or live. Use case IDs instead of real email content in the log. This allows reviewers to compare runs while limiting sensitive data. The report's worksheets are blank templates until actual results are entered; they must not be read as completed test evidence. **Verification detail.** An evidence log should distinguish mocked from live calls and indicate whether the result was observed through the UI or directly at the API. This distinction is essential when a provider account or browser environment was unavailable. **Release consideration.** Evidence folders should contain only artifacts needed to reproduce

a result. Use synthetic message identifiers and remove account details from screenshots. Keep actual credentials and full message bodies out of the report, commit history, and shared diagnostics. **Evidence note.** Keep a stable fixture for each test case so results can be compared after changes. Use fictional examples and store only the evidence required for review. **Operational note.** Use consistent timestamps and timezone notation in evidence records. If a run depends on an external provider, record the provider/model version available at that time without copying credentials. **Handover note.** Retain the test fixture and its expected result so another reviewer can reproduce the same condition after a code change.

I.3 Test case: tone behavior

Tone behavior should be compared across Professional, Casual, Friendly, and no explicit tone. The current base prompt always requests professional wording, which can conflict with the optional tone instruction.

Implementation notes

- Use the same neutral synthetic message for each tone.
- Record whether the result follows tone without changing factual details.
- Compare the React selector with the extension's fixed professional value.

Review and interpretation

Model outputs vary; record provider and model conditions and avoid claiming deterministic wording.

I.4 Test case: model response success

The success case verifies that a valid provider response is parsed and relayed to both clients. A mock response can isolate parser behavior from network availability; a live request separately tests credentials and connectivity.

Implementation notes

- Use a controlled response containing a candidate and text part.
- Confirm the API response is non-empty and the client displays it.
- Check that newlines and punctuation are preserved.

Review and interpretation

Keep mock and live evidence clearly distinguished.

Input and tone expectations. Tone evaluation should keep message facts constant while changing only the requested style. Include the no-tone path because the web form permits omission and the extension sets a fixed value. Confirm whether the service's default instruction conflicts with Casual or Friendly wording. Since model output is variable, acceptance should focus on factual preservation, appropriate style, and user review rather than exact matching to a sample sentence. **Verification detail.** Tone quality should be reviewed by a person using a predefined rubric and more than one synthetic example. Record factual changes and inappropriate commitments separately from stylistic mismatch so the findings can guide prompt changes. **Release consideration.** A tone rubric can score formality, warmth, concision, and fidelity to source facts independently. That helps distinguish a prompt conflict from a model's factual error and gives future prompt revisions a more useful regression signal. **Acceptance detail.** A prompt revision should be evaluated against the same small synthetic case set before and after the change. Record whether factual details, tone, and output length improved or regressed; do not rely on a single favorable generation. **Review note.** Tone precedence should be expressed in both prompt construction and the request schema. If the backend owns the accepted vocabulary, the UI can present those same values while the service rejects unsupported direct requests. **Operational note.** For each

tone, use at least one message that calls for a clear action and one that is only an acknowledgement. This helps distinguish tone compliance from changes in meaning.

I.5 Test case: malformed provider output

Negative parser cases should include missing candidates, missing content, missing parts, empty text, invalid JSON, refusal content, and upstream error bodies. These cases target assumptions visible in the current first-candidate/first-part parser.

Implementation notes

- Prepare one fixture per malformed response shape.
- Verify no unhandled exception or raw stack trace reaches the user.
- Confirm the client receives a stable failure outcome.

Review and interpretation

The current behavior is not asserted until these cases are run.

I.6 Test case: timeout and network loss

A slow or disconnected provider can leave a request waiting and the client in a loading state. Test bounded request duration and guaranteed recovery for both clients.

Implementation notes

- Simulate connection refused, delayed response, and disconnect.
- Measure elapsed time to a user-visible failure.
- Confirm button state and controls recover after failure.

Review and interpretation

Select timeouts based on acceptable user wait and provider behavior; do not add unbounded retries.

Parser and error cases. The response parser's first-candidate assumption should be challenged with a fixture for each absent or malformed layer. A malformed provider payload should become a controlled service error, not an uncaught exception or a string that resembles a normal reply. Client tests then verify that the result panel does not display error text as generated content. Keep raw provider bodies out of the user interface and redact them in logs. **Verification detail.** Parser fixtures should be kept alongside tests and updated when the provider schema changes. A safe failure response should not be confused with a successful generated reply by either client or by a user reading the report. **Release consideration.** When a parser case fails, preserve the minimal synthetic fixture and expected safe outcome. This makes provider-schema changes reviewable and prevents future fixes from silently turning malformed output into apparent success. **Acceptance detail.** A state test should assert the visible button label and enabled state at each transition, not only internal variables. After the request settles, confirm the user can submit a new message without stale errors or old text appearing as the latest result. **Review note.** A retry should begin from a well-defined state and should not leave the prior reply visually indistinguishable from the new result. Include this case in the implementation's state-transition tests. **Operational note.** The expected behavior for an empty provider response should be explicit: no result panel should present it as a successful draft, and the user should receive a safe recovery message. **Handover note.** Keep the error category separate from generated text in both API and client tests.

I.7 Test case: clipboard action

The React copy action uses the browser clipboard API. Verify the result after user activation, then test denied permission and unavailable API behavior.

Implementation notes

- Copy a reply containing line breaks and punctuation.
- Paste into a local text editor and compare exact content.
- Observe whether failure or success is communicated clearly.

Review and interpretation

The source does not currently expose a visible copy confirmation state.

I.8 Test case: Gmail action presence

This case verifies that the extension action appears once in the intended Gmail compose surface and remains correctly associated with the active message.

Implementation notes

- Open a new compose, a reply, and multiple compose windows.
- Observe control count after repeated DOM mutations.
- Navigate between messages and confirm action scope.

Review and interpretation

The supplied Work 13 image is visual evidence of a compose view with the AI Reply action, not this complete test.

Client and extension cases. Clipboard tests should preserve line breaks and punctuation, while Gmail tests should verify the selected message, active editor, and existing draft. These workflows should be tested independently: a working copy action does not prove correct extension extraction, and a visible extension button does not prove insertion. Use synthetic content and retain only the minimum screenshot or result needed to document each case. **Verification detail.** Preserve evidence of both the intended output and the unchanged surrounding draft. Remove account identifiers from screenshots and use a test browser profile so personal mailbox state is not copied into the report. **Release consideration.** The Gmail test should also verify that a failed insertion leaves the old draft intact and does not change focus to an unrelated compose window. Keep the screenshot and text comparison paired to the same test case. **Acceptance detail.** Before deployment, exercise one request from each client against the environment-specific API and verify the origin policy. A production URL in the browser is not sufficient evidence until transport, credential handling, and error recovery are checked there. **Review note.** A deployment check should use a configuration checklist and a synthetic request, then verify the resulting traffic in the target browser. Do not rely on a successful local build as proof that HTTPS and origin settings work remotely. **Operational note.** After copying, preserve the displayed text and provide a confirmation state only when the clipboard operation succeeds. If permission fails, keep a manual selection path available. **Handover note.** The browser check should include both clipboard permission granted and denied, using the same reply text.

I.9 Test case: Gmail extraction and insertion

Verify that the extracted text matches the intended conversation context and the returned reply is inserted at the correct location in the draft. The extension must not trigger Send or overwrite unrelated text.

Implementation notes

- Use a synthetic message and a draft with existing text.
- Compare extracted content with the intended message only.
- Confirm inserted text placement and preserved draft content.

Review and interpretation

Use a test mailbox and remove identifying information from saved evidence.

I.10 Test case: origin and credential handling

Review browser assets, requests, source files, logs, and screenshots for credentials and sensitive content. Verify cross-origin behavior against the intended deployment origin.

Implementation notes

- Search the client build and extension files for provider keys.
- Inspect request URLs and sanitized server logs.
- Attempt an unapproved origin and record the observed policy.

Review and interpretation

Source currently appends the key to the configured Gemini URL, so production logging behavior needs explicit review.

Credential and origin evidence. Credential checks should inspect source, production bundles, extension packages, request URLs, and sanitized logs. Origin tests should use both allowed and disallowed clients and record the actual CORS behavior. Since the current provider key is appended to a URL, URL redaction deserves special attention. Do not paste secrets into the worksheet; identify only the configuration source and whether the review passed. **Verification detail.** A secret review should include URL telemetry and exception traces because query parameters can be captured outside application logs. Rotate any credential that was ever included in a shared artifact or diagnostic record. **Release consideration.** A credential rotation procedure should be documented even when no exposure is known. Review URL redaction in the proxy and monitoring layer, where application-level redaction may not remove query parameters automatically. **Acceptance detail.** A release review should confirm that open risks have owners and that the user notice matches the configured provider and retention behavior. Keep the acceptance decision tied to the exact package and source revision being demonstrated. **Review note.** Release restrictions should be stated near the acceptance decision, such as tested browser scope, test data policy, or unresolved integration cases. Reassess them after each relevant fix rather than carrying them forward automatically. **Operational note.** A browser capture should be reviewed at normal zoom before inclusion. Remove names, account icons, and message previews even when the test data is synthetic, since screenshots can expose unintended context.

I.11 Defect severity and triage

A consistent severity scheme helps the team prioritize issues without overstating prototype defects. Severity should reflect user impact, privacy, data loss, and availability.

Implementation notes

- Critical: secret exposure, unintended sending, or cross-user data disclosure.
- High: core workflow fails or overwrites a user's draft.
- Medium: recoverable failure or significant accessibility barrier.
- Low: cosmetic issue with a clear workaround.

Review and interpretation

Agree severity criteria before triage and record the rationale for each assignment.

I.12 Test evidence and sign-off form

A sign-off form should make clear what was run, what was observed, and what remains open. Planned cases should never be counted as passing evidence.

Implementation notes

- Fields: case ID, revision, environment, observed result, status, evidence path, defect ID.
- Reviewer confirms no secrets or private email data are included.
- Approver records date and any accepted residual risk.

Review and interpretation

Use 'not run' or 'blocked' where applicable and preserve the evidence needed for later review.

Triage and sign-off. A sign-off should distinguish defects fixed from accepted residual risks and cases not run. Severity should be based on user impact and data exposure, with a clear rationale. A release reviewer should confirm that evidence matches the same source revision and that screenshots contain no private information. Blank or blocked fields should remain visible until someone runs the check; a completed-looking worksheet is not a substitute for execution. **Verification detail.** Before sign-off, reconcile every claimed result with its evidence and list open defects in plain language. Reviewers should be able to tell which items were executed, which were blocked, and which remain recommendations. **Release consideration.** Approvers should see open items before accepting a demonstration. A concise sign-off record can identify restrictions, such as synthetic content only or a single tested browser version, without implying that unresolved cases passed. **Evidence note.** The sign-off record should be dated and tied to the exact revision tested. If a critical case is blocked, restrict the demonstration accordingly and identify the follow-up owner. **Operational note.** A case that is not run should include the reason and the condition needed to unblock it. Keep the status visible in the report until the follow-up test is completed. **Handover note.** The handover reviewer should confirm that no case marked passed relies only on a source-code inspection or an image. Keep the evidence path available to the supervisor and record any redaction performed before sharing. **Final check.** Assign an owner and due date to every open high-impact defect before sign-off.

APPENDIX J - DEPLOYMENT, QUALITY, AND HANDOVER REFERENCE

J.1 Local setup checklist

A reproducible local setup should follow the actual repository layout and keep provider secrets out of checked-in files. The supplied README describes Node.js, Java, Maven, and Gemini configuration, but its sample paths should be checked against the repository.

Implementation notes

- Confirm frontend, backend, and extension directories.
- Install dependencies and verify runtime versions.
- Set local GEMINI_URL and GEMINI_KEY securely before startup.

Review and interpretation

A clean setup should be repeated from a fresh checkout before handover.

J.2 Configuration matrix

Configuration values should be separated by environment so a local URL cannot accidentally ship in a production client. The current React and extension sources use a local endpoint, and the extension also has a fixed tone.

Implementation notes

- Development: local API, test credentials, synthetic content.

- Staging: protected provider secret, HTTPS, test mailbox.
- Production: managed secret store, restrictive origin policy, monitored quotas.

Review and interpretation

This matrix describes recommended environments, not existing deployment environments.

Environment-specific setup. Setup instructions should begin with a clean checkout and identify each working directory, runtime, and required environment value. Validate the backend before launching either client, then test one synthetic request. Keep production and staging URLs out of the local-only extension bundle unless intentionally configured. Clear setup documentation reduces hidden reliance on a developer's machine and exposes mismatches between the README and actual source structure. **Verification detail.** Write the setup sequence so a new maintainer can reproduce it without undocumented machine state. Include safe example values, expected startup signals, and a smoke-check using fictional content. **Release consideration.** A fresh-checkout setup should verify the frontend, backend, and extension separately before exercising the end-to-end flow. Record successful build/start signals and configuration names, not machine-specific paths or secret values. **Evidence note.** The setup guide should state when the backend is ready before the user opens either client. This reduces false client-side errors caused by startup order. **Operational note.** Keep the release checklist short enough to use, but specific enough to cover build identity, required configuration, origin policy, and a synthetic smoke request. **Handover note.** Record the clean build and smoke-check date so setup drift can be identified later. **Final check.** Confirm that the smoke request uses fictional content and returns the expected client state.

J.3 Release gate

A release gate should require code, security, integration, and documentation evidence before the prototype is used with real messages. A gate also prevents screenshots or prose from implying a live feature that was not exercised.

Implementation notes

- Builds are reproducible and configuration is externalized.
- API validation, failure mapping, and basic client recovery are verified.
- Extension behavior is checked against the target browser and Gmail view.
- Privacy notice and user review responsibility are clear.

Review and interpretation

Record any waived item and its owner; do not silently call the release production-ready.

J.4 Backup and rollback plan

A rollback plan helps restore a known version if a release changes prompt behavior, breaks Gmail selectors, or exposes a service issue. The current source does not contain a deployment pipeline, so these are handover recommendations.

Implementation notes

- Tag the source revision and preserve build artifacts.
- Keep the previous known-good backend and extension package available.
- Document how to disable the extension or revert the service configuration.

Review and interpretation

Test rollback steps in a non-production environment before relying on them.

Release and rollback evidence. A release gate should identify required checks, known defects, and the person responsible for the decision. Preserve a previous package and configuration so a failed browser or provider change can be reversed. The repository does not show a deployment pipeline, so a release checklist and manual rollback procedure are practical first controls. Test the rollback steps outside production before relying on them. **Verification detail.** Record the exact previous and candidate revisions and test the disable/rollback procedure. A fallback should restore a known package without exposing provider keys or destroying user drafts. **Release consideration.** Keep rollback limited to versioned artifacts and configuration references; it should not clear drafts or browser data. Record the procedure for disabling the extension and restoring a previous backend package. **Evidence note.** Record the artifact identifier and configuration category needed to restore the previous release. Do not copy secrets into rollback instructions or report appendices. **Operational note.** Rollback instructions should state the order of changes and the confirmation that the previous version is active. Avoid steps that clear local storage or alter user drafts. **Handover note.** A release note should state the changed components and the checks rerun for each one. **Final check.** Keep the previous artifact available until the candidate release is accepted.

J.5 Operational ownership

A live service needs a named owner for provider credentials, endpoint availability, browser compatibility, incident response, and privacy requests. No production ownership structure was supplied with the project files.

Implementation notes

- Assign a technical contact for API and deployment issues.
- Define a process for credential rotation and quota review.
- Provide a support route that does not ask users to send private email text.

Review and interpretation

Record role names and contact details in the deployment-specific handover, not this generic report.

J.6 Privacy notice checklist

Before sending real email content, users need a clear explanation of what is transmitted and to which provider. The source code alone cannot establish data retention terms, contractual controls, or hosting logs.

Implementation notes

- Explain that message text and selected tone reach the backend and model provider.
- State applicable retention, access, and deletion practices from verified deployment terms.
- Offer guidance to omit sensitive personal or confidential information.

Review and interpretation

Have the notice reviewed against actual provider and institutional requirements.

Operations and privacy. Operational monitoring should track availability, latency, and sanitized error categories without collecting the email body. Assign owners for credential rotation, endpoint health, provider quota, extension compatibility, and privacy requests. A user-facing notice should describe the actual provider path and verified handling terms. No deployment ownership or retention policy was included with the supplied files, so those decisions remain open for the target environment. **Verification detail.** Privacy ownership includes handling access requests, incident review, provider configuration, and retention settings. These responsibilities must be assigned from the actual deployment context; this report intentionally leaves personnel and policy facts open. **Release consideration.** Review actual

retention behavior across the backend, proxy, provider account, and local evidence store. Update the notice whenever a deployment changes a recipient, logging setting, retention period, or model provider. **Evidence note.** For service incidents, define how to disable the extension or web client without deleting or modifying a user's existing draft. **Operational note.** A continuity plan should identify who can disable the service if a provider issue or selector regression creates unsafe behavior. Keep the manual email workflow available. **Handover note.** For each operational owner, identify the decision they can make during an incident and the information needed to make it safely. **Final check.** Document the contact route for reporting an incident without sending private message text.

J.7 Known limitations register

The report should present current boundaries and future work together so that prototype behavior is not mistaken for a validated service. These items were observed by reviewing the supplied source.

Implementation notes

- Limited server-side input validation and provider error mapping.
- Response parser assumes a first candidate and first text part.
- Extension relies on Gmail DOM selectors and fixed configuration.
- No supplied evidence of executed end-to-end tests or production deployment.

Review and interpretation

Update this register when a limitation is fixed and verified.

J.8 Glossary and handover summary

A concise glossary helps a reviewer understand the terminology used across the architecture and report. The handover summary records the component boundaries and next actions.

Implementation notes

- Client: React page or Gmail content script that begins a request.
- Backend: Spring Boot API and Gemini service integration.
- Draft: generated text presented for human review.
- Handover: configuration, build, validation, and ownership materials.

Review and interpretation

The next engineering priorities are request validation, response handling, secure configuration, and verified client integration.

Known limitations and handover. The handover should preserve the distinction between implemented source behavior and recommended future work. The current project shows the basic generation path, but validation, robust error mapping, secure deployment, and reliable Gmail integration remain open. Record these as actionable issues with evidence required for closure. This provides a practical starting point for the next maintainer without overstating readiness or duplicating generic product claims. **Verification detail.** Keep limitations visible in the next release note until each item has implementation and test evidence. This gives a reviewer a concrete handover list and prevents prototype capabilities from being described as mature service behavior. **Release consideration.** A limitation can be closed only after a change is merged into the reviewed source and a relevant check passes. Preserve the old status in release history so future readers can see when a claim changed and what evidence supports it. **Evidence note.** A handover note should list the next concrete work items in priority order and point to the evidence needed to close each limitation. **Operational note.** Review the limitations with the project supervisor before final submission and ensure the report does not imply that suggested controls have

already been implemented. **Handover note.** Carry forward only limitations that remain unresolved at the reviewed revision, and link each one to a follow-up task or evidence requirement.

APPENDIX K - API IMPLEMENTATION REFERENCE

This appendix summarizes the API contract that connects the checked-in React client, Spring Boot service, Gemini provider, and Gmail content script. It records current source behavior and labels recommendations as future work.

K.1 API route and request fields

The shared route is POST /api/email/generate. The request DTO contains emailContent and tone. In the React client, tone is optional and the visible options are Professional, Casual, and Friendly. The extension currently supplies a fixed professional value. The controller binds the JSON body and delegates to the generation service. The server does not show field-level validation annotations or a maximum request size in the supplied files.

K.2 Provider request and response

The service creates a provider payload with contents, parts, and text. The prompt includes a reply instruction, a tone instruction when provided, and the original email. The response parser selects the first candidate and first text part. It returns raw text to the controller; the current client contract is not a typed success/error envelope.

K.3 Contract checklist

Before releasing additional clients, document field types, blank-input response, tone allow-list, maximum content length, timeout, provider failure mapping, and whether the API response stays raw text or becomes a JSON object. Keep both clients and examples synchronized when the contract changes.

K.4 Validation scenarios

Before release, exercise valid input, missing emailContent, whitespace-only content, unsupported tone, and content over the intended size limit. Reject invalid requests before prompt construction and define the observed HTTP result. Test provider timeout, empty output, and malformed JSON with deterministic fixtures; then verify that both clients recover without displaying an error as generated text. Keep request examples fictional, record whether the provider was mocked, and synchronize any contract change across React and the Gmail extension.

K.5 Evidence and compatibility

Record whether each contract case was run against a mock or a live provider, and attach the source revision and client path to the result. If the raw text response is replaced by a structured object, update and recheck the web page and Gmail extension together. A contract example should describe observed behavior only; proposed status codes and validation rules remain recommendations until implemented.

Element	Observed behavior	Release note
Route	POST /api/email/generate	Use HTTPS outside local development
Request	emailContent, tone	Validate length and tone server-side
Response	Raw reply string	Define stable errors before versioning

Review and interpretation

The route and field names are source-derived. The last-column items are recommended hardening work and should not be read as already implemented.

APPENDIX L - IMPLEMENTATION AND RELEASE ANALYSIS

L.1 Request contract and boundary validation

The current request crosses two boundaries: a browser client submits JSON to Spring Boot, and the backend submits a prompt to Gemini. A reliable contract should make the checks at each boundary explicit instead of relying on the React form alone.

Implementation notes

- Require a non-null emailContent string and reject whitespace-only input on the server.
- Define an allow-list for tone and decide whether omission maps to a neutral default.
- Set a documented maximum request size before any provider call.
- Keep client-side checks for immediate feedback, but treat backend validation as authoritative.

Review and interpretation

The checked-in DTO contains emailContent and tone without validation annotations. Add contract tests for accepted and rejected payloads before changing either client.

L.2 Success, failure, and response semantics

The controller currently returns a raw string and the service can also return an error string after a parsing exception. That makes a generated reply and a service failure difficult for clients to distinguish. A stable contract should separate successful output from failure metadata.

Implementation notes

- Choose either a documented text response or a JSON object with a reply field; update both clients together.
- Map invalid requests, provider rejection, timeout, empty output, and unexpected parsing failures to distinct status categories.
- Return user-safe messages while keeping implementation details in protected diagnostics.
- Ensure the React and extension clients always restore their loading state.

Review and interpretation

These are proposed contract improvements, not behaviors currently verified in the source. Preserve backward compatibility deliberately if an API response shape changes.

Request acceptance criteria. For the request contract, a useful acceptance table should include missing fields, wrong JSON types, blank content, unsupported tones, boundary-length content, and valid input. Each rejected case should identify the HTTP result and confirm that the provider was not called. A valid case should preserve the exact user text passed to prompt construction, subject to documented normalization rules. These checks make the contract observable to both the React client and the

-
-

extension. **Release consideration.** The boundary cases should be defined consistently for both clients. If the UI blocks a blank value but the API accepts it, an extension or third-party caller can still generate ambiguous provider requests. Keep the expected status and user message in a contract fixture. **Evidence note.** The server-side contract should be tested with direct requests as well as client submissions. Keep the browser checks for usability, but do not rely on them to enforce accepted values. **Operational note.** Contract tests should use the same field names and casing as the clients. This catches drift between JSON binding and frontend serialization before a live provider call is made.

L.3 Prompt composition and tone precedence

The service constructs a professional-reply instruction and appends a second instruction when a tone is supplied. Since Professional, Casual, and Friendly can conflict with the unconditional base instruction, prompt construction should have one clear source of tone authority.

Implementation notes

Build a neutral base instruction and add exactly one validated tone directive. Keep

the request's original message visibly separated from system guidance.

- Specify concise output and no invented subject line if that remains the product behavior.
- Use a neutral default when tone is absent and document it for both client paths.

Review and interpretation

A deterministic prompt-builder unit test can verify the exact prompt without a live provider. Tone quality still needs model evaluation under recorded conditions.

L.4 Untrusted message content and output review

An incoming email is user-provided data, not trusted instruction for the application. It may contain quoted text, signatures, or instructions directed at an AI system. The generated reply is also untrusted output because it can omit context or introduce unsupported details.

Implementation notes

- Delimit the source email and instruct the model to treat it as context rather than governing instructions.
- Do not let generated content trigger application actions or email sending.
- Keep the reply editable and visibly subject to review before use.
- Ask users to verify names, dates, amounts, links, commitments, and sensitive claims.

Review and interpretation

-
-

Prompt safeguards reduce ambiguity but cannot guarantee accuracy. Human review remains a product requirement and should be stated at the point of use.

Client compatibility. A response-shape change is a coordinated release, not a backend-only edit. The React code currently treats response data as text or serializes it, while the extension uses the response to update a Gmail editor. If a JSON envelope is adopted, test successful display, error display, clipboard content, and insertion behavior in both clients. Keep an explicit compatibility example in the API reference so future clients do not infer behavior from implementation details. **Release consideration.** Status mapping should avoid leaking provider keys, raw URLs, stack traces, or private prompt content. The client needs enough information to tell the user whether to correct input, retry later, or contact support, but not the underlying secret or server exception. **Evidence note.** A safe failure response should be documented with status, code, and user message once implemented. Until then, describe current source behavior and keep proposed mappings clearly marked. **Operational note.** Clients should distinguish a retryable provider outage from a request that must be corrected by the user. Use stable response codes rather than parsing human-readable text.

L.5 Gmail DOM lifecycle and mutation handling

The extension uses a MutationObserver because Gmail can add and replace compose elements after the page initially loads. Observer callbacks can receive batches of changes, and repeated callbacks can cause duplicate actions unless insertion is idempotent.

Implementation notes

Check each relevant compose container rather than assuming the first changed node is the target.

Mark an injected control or use a scoped query to prevent duplicates.

- Disconnect or narrow observers when a compose surface is removed.
- Handle navigation, collapsed drafts, and newly opened compose windows explicitly.

Review and interpretation

The supplied selector helpers return from their first loop iteration, so fallback behavior needs correction. Verify lifecycle cases in a test mailbox without risking unsaved drafts.

L.6 Compose scoping and draft preservation

A Gmail page can contain several conversations and compose windows at once. Reading the first matching message or inserting into the first editable textbox can associate a reply with the wrong context. The action should be scoped from the button's own compose or conversation container.

Implementation notes

- Resolve the active conversation from a stable ancestor before extracting message text.
- Pair the AI Reply control with the editor in the same compose surface.

-
-
- Preserve existing draft content and define whether insertion replaces a selection or appends text.
- Never locate or activate the Send control as part of generation.

Review and interpretation

These rules address data association and accidental overwrite risk. The current screenshot proves button placement only; it does not prove these behaviors.

Prompt regression conditions. Prompt regression cases should verify that the chosen tone appears once, the source message is delimited, and no unsupported fact is requested or added. Include a no-tone case and an email containing quoted or instruction-like text. Store prompt fixtures without personal information and compare construction separately from model output. For live evaluations, record the provider model and date because provider behavior can change even when the application source does not. **Release consideration.** Prompt fixtures should be reviewed when the base instruction or tone vocabulary changes. A regression comparison can test exact assembly, while human review of model output should be repeated for representative cases because a fixture alone cannot guarantee language quality. **Evidence note.** Include examples where the incoming message contains quoted instructions or a signature. The prompt should preserve useful context without treating those strings as system directions. **Operational note.** A prompt review can compare the generated request text, but it cannot prove the model will follow it. Maintain that distinction in test evidence and in the report's conclusions.

L.7 React request state and recovery

The React component stores content, tone, result, loading, and error independently. Independent state is simple for a prototype but can allow stale output or error text to remain visible across successive requests if transition rules are incomplete.

Implementation notes

On a new request, clear or label the previous result so it is not mistaken for the new reply.

Set loading before the request and reset it in a finally path for every outcome.

- Define whether edits to input invalidate the displayed result.
- Prevent duplicate submissions while a request is active and preserve the user's entered text after failure.

Review and interpretation

A small state-transition table should guide UI tests for idle, invalid, submitting, success, and error states. Do not claim cancellation or retries until they exist in code.

L.8 Accessible feedback and keyboard operation

-
-

The web form uses Material UI controls, while the Gmail action is created directly in the page. A visually clear action can still be difficult to use if it lacks a programmatic name, keyboard activation, visible focus, or announced status changes.

Implementation notes

- Use semantic buttons and labels for both clients.
- Expose loading and error changes to assistive technology without repeatedly interrupting the user.
- Check contrast, zoom, focus order, and operation without a pointer.
- Keep the inserted reply editable and provide a clear recovery path if insertion fails.

Review and interpretation

Accessibility checks must be performed in the running interface. Component-library usage alone does not establish conformance.

Human review checkpoints. The review step should be visible before the user copies or inserts the proposed text. The interface can remind the user to check factual details and tone without presenting the reminder as a guarantee. Any future action that changes a draft should remain reversible and should not send a message. A user should be able to discard the result or edit it in the mail client; generation must not remove the original source message or existing draft. **Release consideration.** The review reminder should be concise and placed near the generated text or before insertion. Avoid a design that users must dismiss repeatedly. The goal is to make editing and verification part of the normal workflow, not to imply that model output is automatically safe. **Evidence note.** The user should remain able to edit, reject, or discard the draft. A review reminder can support that choice, but the product should not block normal editing or imply perfect accuracy. **Operational note.** A user should see that the text is generated before it is inserted. The reminder should support informed review without interrupting the flow or appearing as a claim of guaranteed safety.

L.9 Data inventory and trust boundaries

The principal user content is emailContent, accompanied by a tone value. The web client sends this information to the backend, which includes the email text in a provider prompt. The model reply travels back through the service to the client and may then be copied or placed in a Gmail draft.

Implementation notes

Identify email text and generated text as sensitive by context, even when no account data is stored locally.

- Document each transfer, recipient, transport, and storage point for the deployment actually used.
- Minimize diagnostic collection and avoid retaining message bodies by default.
- Treat browser page content and model output as untrusted at their respective boundaries.

Review and interpretation

No database or persistent reply-history subsystem was identified in the supplied source, but deployment logs and provider processing still require review.

L.10 Credential protection, logs, and privacy notice

The Gemini key is configured on the backend but appended to the provider URL in the current service. URLs can appear in diagnostics or monitoring systems, so secret placement and log redaction need explicit attention. A user notice should describe the actual message-processing path.

Implementation notes

- Use the provider's supported authentication mechanism and protect secrets outside the repository.
- Redact query parameters and request bodies from logs, traces, and error reports.
- Restrict CORS and extension host access to the deployment's required origins.
- Explain that email text is sent to the backend and external model provider, using verified retention terms.

Review and interpretation

A source-level review cannot establish provider retention, contractual protections, or production logging behavior. Verify those facts in the target environment.

Observer test matrix. A browser regression matrix should include a page loaded before the extension, a compose window opened after load, a collapsed and reopened draft, navigation to another conversation, and several simultaneous compose windows. Record which mutation triggered insertion and whether the observer added one control or duplicates. Test missing primary selectors to prove that each fallback is reached. These cases directly exercise the dynamic-page design rather than only checking that the script loads. **Release consideration.** A browser test should confirm that observer cleanup does not remove Gmail controls and that repeated page updates do not multiply listeners. Keep a short screen recording or redacted screenshot only when it materially helps reproduce a lifecycle defect. **Evidence note.** Testing should include removing the compose surface while a request is active. The extension must not write the response into a replacement or unrelated editor when the original surface disappears. **Operational note.** If an active request cannot be cancelled, the extension should still ensure its eventual response is not written to a different editor. A cancellation design can be considered separately from safe completion.

L.11 Environment configuration and deployment separation

The React and extension sources currently use a local HTTP endpoint, while the backend reads provider values from environment placeholders. Local demonstration, staging, and public deployment need separate configuration so that development URLs and secrets cannot be carried into a release accidentally.

Implementation notes

- Make the API base URL environment-specific in both clients.
- Use HTTPS outside local development and a protected secret store for provider credentials.
- Set explicit origins, request size limits, and timeouts at the service boundary.
- Provide example configuration with placeholders rather than real keys.

Review and interpretation

The source does not show a production deployment. Treat deployment profiles and operational ownership as work to complete, not as existing capabilities.

L.12 Availability, latency, and resource behavior

A generation request depends on the browser, backend, network, provider availability, and provider quota. The service calls a reactive client and then blocks for a result, so request duration and thread usage should be measured under realistic conditions before selecting deployment capacity.

Implementation notes

- Set connect and response timeouts and return a controlled failure when they expire.
- Use bounded concurrency and request-size controls to protect the service.
- Avoid automatic retries that can create duplicate provider cost without a clear policy.
- Measure client wait time and backend/provider time separately using synthetic requests.

Review and interpretation

No latency or load measurements were supplied. Capacity and performance claims should remain open until an instrumented test records them.

Insertion acceptance evidence. The insertion check should capture the draft before and after generation using synthetic text and compare unchanged portions. Confirm the active editor belongs to the same compose surface as the AI control and that the extension restores its state after failure. A visual screenshot is useful for position and focus, while a text comparison verifies content preservation. Keep both kinds of evidence separate and never include a real message body in the institutional report. **Release consideration.** Define insertion semantics before implementation: replace selected text, append at cursor, or fill an empty editor. The chosen behavior should be visible and tested with pre-existing draft content so that users can predict the result. **Evidence note.** Agree on insertion semantics with users before implementing them. If the text replaces a selection or existing content, show that behavior clearly and preserve a way to undo the change. **Operational note.** The insertion test should preserve punctuation, line breaks, and existing text. Record whether insertion replaces a selection or appends at the cursor so that future UI changes do not alter the behavior silently.

L.13 Test architecture and isolation

Validation can be split into deterministic component checks and environment-dependent integration checks. Prompt construction and parsing can be tested with fixtures; network credentials, provider availability, browser DOM behavior, and user interaction require separate evidence.

Implementation notes

- Unit-test request validation, tone selection, prompt assembly, and provider-response parsing.
- Controller-test status codes and response shape without external network access.
- Use a mock provider for reproducible client-to-service cases.
- Run live-provider and Gmail checks only with synthetic or consented content.

Review and interpretation

Keep test results tied to a source revision and identify which layers were mocked. A passing unit test is not evidence of Gmail insertion or provider availability.

L.14 Test data and evidence integrity

Email examples can contain personal, business, or authentication data. The project needs a test-data policy that allows useful workflow checks without placing real mailbox content in reports, logs, or screenshots.

Implementation notes

- Use short synthetic messages representing acknowledgement, scheduling, clarification, and update tasks.
- Do not include real names, email addresses, credentials, financial details, or confidential attachments.
- Capture only the interface state needed to demonstrate the workflow.
- Record the environment, case ID, observed result, and evidence path without retaining the full message body.

Review and interpretation

The two included images show the initial web state and Gmail action placement. They do not document a live model response or a measured test result.

State transition cases. For each state, define the visible control, stored value, and next action. An empty request should not enter loading; a valid request should disable duplicate activation; success should expose a new reply; failure should retain the source input; and a subsequent request should not inherit a stale error. The current implementation stores state in one component, so these rules can be implemented as explicit transition logic or tested through user-visible behavior. **Release consideration.** For a new request, decide whether the previous reply is retained with a timestamp or cleared. Either choice can work if it is unambiguous; a stale draft presented without context risks being copied as if it came from the latest message. **Evidence note.** Model the prior result as stale when the source message changes. Clear it or label it so the user cannot confuse a reply to an earlier message with the current draft. **Operational note.** When the source input changes, either clear the old result or label it as belonging to the previous input. Verify that the visual state makes this relationship obvious.

L.15 Release acceptance and residual risk

A release decision should be based on evidence for the complete request path and on explicit treatment of the remaining privacy, reliability, and Gmail-integration risks. A local prototype can be suitable for a controlled demonstration while still being unsuitable for real confidential mail.

Implementation notes

- Require validated input, stable error outcomes, secure configuration, and recoverable client states.
- Verify Gmail control placement, correct message scoping, draft preservation, and absence of send activation.
- Review the notice and provider-processing terms for the target deployment.
- Record unresolved issues, owners, and any demonstration restrictions.

Review and interpretation

Do not label the system production-ready based on source inspection or screenshots alone. Revisit acceptance when fixes and test evidence are available.

L.16 Maintenance and change traceability

Changes to provider models, prompts, Gmail selectors, dependency versions, and API schemas can alter behavior even when the interface looks unchanged. A maintainable handover should connect each release artifact to the source revision and to the evidence used for acceptance.

Implementation notes

- Record commit, build date, runtime versions, configuration category, and package outputs.
- Version prompt and API changes with focused regression cases.
- Review extension selector compatibility and provider response assumptions when upstream formats change.
- Preserve rollback information and remove obsolete build artifacts only after handover.

Review and interpretation

No formal release or change log was supplied. Introduce traceability as a future control and keep report claims aligned with the exact revision reviewed.

Operational handover fields. A deployment handover should name the API owner, provider-credential owner, extension release owner, incident contact, and process for disabling a broken integration. Record how to rotate credentials, change the endpoint, review provider quota, and roll back a client build. These are deployment-specific values and should not be invented in a generic report. The checklist is complete only when the relevant team fills in verified contacts and procedures. **Release consideration.** Operational fields should be completed by the actual deployment owner, not inferred from the project source. Include escalation and disablement instructions that can be followed during a provider outage or a Gmail compatibility failure. **Evidence note.** Review provider endpoint, secret source, logging settings, and browser permissions whenever the deployment environment changes. The handover owner should verify each setting from the deployed package. **Operational note.** Keep the handover record current after provider, browser, or hosting changes. A contact list and rollback package are useful only if they match the current deployment.